UPPSALA
UNIVERSITET

IT mDV 26 013
Degree project 30 credits
June 2026

Random Number Generator from Aggregated Cryptocurrency Prices

with an Application to Secure Password Generation

Yujia (Kira) Liu

Master's Programme in Computer Science

UPPSALA
UNIVERSITET

Random Number Generator from Aggregated Cryptocurrency Prices
with an Application to Secure Password Generation

Yujia (Kira) Liu

Abstract

Randomness is a basic requirement for many computational tasks, especially in security, but a reliable physical source of randomness is not always conveniently available on ordinary devices. This thesis studies a concrete research question: can passively observed public cryptocurrency trade data, after statistical aggregation, validation, and cryptographic conditioning, provide usable entropy for strong password generation? The cryptocurrency market trades continuously and produces high-frequency public data, which makes it a suitable test case for this question. The main difficulty is that raw price changes are not independent: market microstructure makes the signs of neighbouring price changes strongly correlated, so the data cannot be used directly as random bits.

This thesis builds and evaluates a pipeline that runs from market prices to password output. Price changes are first encoded as bits, then aggregated on different time bases, screened by statistical tests, and combined across several assets. The bit streams that pass validation are then passed into a standard cryptographic conditioning step that produces the final passwords. The experimental data cover five high-liquidity USDT spot pairs over a span of 15 months.

The experiments give a positive answer with some limitations. Raw per-trade price changes fail the statistical checks in every month. After aggregation into one-second time windows, most short-range dependence is removed, but the output of a single asset still retains some higher-order structure. Combining several assets reduces this residual further: three configurations stay stable on the held-out validation months, while one four-asset configuration fails because its bias drifts outside the calibration range. The final prototype uses the stable configurations to generate 16-character passwords; measured by search-space entropy, each password carries about 98 bits of strength, well above the roughly 30 bits typical of user-chosen passwords and above the 80-bit working benchmark for strong random passwords adopted in this thesis. The overall conclusion is limited but practical: under a passive statistical setting and prototype-level evaluation, public cryptocurrency data can serve as entropy input for password generation after aggregation, validation, and cryptographic conditioning.

Faculty of Science and Technology
Uppsala University, Uppsala

Supervisor: Andrey Shternshis Subject reader: Parosh Abdulla
Examiner: Lars-Henrik Eriksson

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

Random numbers are used in many areas, including cryptography, digital signatures, data anonymisation, and simulation. They are one of the most basic building blocks for system security and correctness.

There are two common types of random number sources. The first type is the pseudo-random number generator (PRNG), which uses a deterministic algorithm. It starts from a short initial value, called a *seed*, and expands it into a long sequence of numbers that look random. As long as the seed is kept secret, the output is unpredictable. The catch is that the seed itself must be genuinely hard to guess — what is technically called a *high-entropy* value, meaning it carries enough true unpredictability that an attacker cannot reproduce it. A PRNG therefore does not remove the need for true randomness; it only shifts that need to producing a good seed.

The second type is the true random number generator (TRNG), which uses physical processes such as electronic noise — for instance, the on-chip RDRAND/RDSEED instructions in modern Intel CPUs (Mathew et al., 2012) — or quantum sources such as photon detection and vacuum-fluctuation measurement, which are surveyed in detail by Herrero-Collantes and Garcia-Escartin (2017) and have been commercialised as products like ID Quantique's Quantis. TRNGs provide real unpredictability, but they require special hardware, are expensive to deploy, and are not easy to access on ordinary user devices.

In recent years, a middle-ground approach has attracted research interest: whether randomness can be extracted from high-entropy natural data sources that already exist and are publicly accessible. Candidates studied in the literature include blockchain data such as Bitcoin block hashes (Bonneau et al., 2015; Pierrot and Wesolowski, 2018) and financial market data; the latter is the focus of this thesis. It is produced by a large number of participants at very high frequency, contains microstructure noise that is hard to model, and shows complexity that is hard to compress beyond simple statistical structure. Several previous works have explored this idea under the name *financial randomness beacon* (Chiba and Ichikawa, 2024; Clark and Hengartner, 2010; Landis and Bonneau, 2025), trying to turn public market data into a candidate randomness source.

However, using the sign of price changes directly as a random bit stream does not work. Many studies have shown that high-frequency trading sequences look balanced at the marginal level (for example, the up/down ratio is close to 1:1), but neighbouring symbols are strongly correlated at short lags. A common cause is microstructure noise such as the bid-ask bounce (Cont, 2001;

Shternshis and Marmi, 2025). To turn this kind of "looks random but is conditionally predictable" data into a usable random bit stream, a pipeline is needed. In this thesis the pipeline is: take market prices, encode the sign of price changes as bits, aggregate neighbouring observations to reduce short-range dependence, test the resulting bit streams, and only then pass acceptable streams into a cryptographic conditioning step.

This pipeline is not specific to one market. In principle, it can be applied to many financial data sources. This thesis focuses on the cryptocurrency spot market for practical reasons. Compared to traditional stock markets, the cryptocurrency market runs 24 hours a day without exchange closures, and its public APIs (such as Binance) allow direct download of every trade, with no broker batching or routing layer in between. Market activity is therefore directly visible to end users. These properties make cryptocurrency data a useful test case for the pipeline. At the methodological level, the later experiments compare different aggregation axes and choose a suitable aggregation granularity.

For the application target, this thesis uses the accepted streams for *strong password generation*. The literature gives a context for this choice: an early large-scale study of web password habits (Florêncio and Herley, 2007) and a later guessing-corpus analysis (Bonneau, 2012) both report that user-chosen passwords often have low entropy, around 20–30 bits, and randomly generated passwords are one of the countermeasures discussed in that literature. This thesis therefore uses market-derived randomness as input to a standard cryptographic conditioning step that produces passwords, giving an application target with a clear scope that can be checked end to end within the thesis.

1.2 Brief Literature Review

The literature directly related to this work falls into two lines. This section locates the thesis relative to them.

Financial and public randomness. Several works use public financial data, and by analogy on-chain data, as a randomness source. Clark and Hengartner (2010) turned the Dow Jones 30 into a verifiable public beacon; Bonneau et al. (2015) and Pierrot and Wesolowski (2018) assessed the feasibility and the malleability of Bitcoin block hashes as a similar public source; Chiba and Ichikawa (2024) combined cryptocurrency prices with a Linear Feedback Shift Register (LFSR, a basic building block that produces long pseudo-random bit streams from a short internal state) for bit extraction; and Landis and Bonneau (2025) evaluated the security cost of a financial beacon under an active-attacker model. These works cover the protocol level, the bit-extraction level, and the adversarial-security level, but none of them connects the full path from "raw data" to "a verifiable application prototype".

Statistical market randomness. A second line studies the statistical properties of financial time series themselves, from a randomness and aggregation point of view. Shternshis and Marmi (2025) introduced an entropy-based predictability test D based on the Kullback–Leibler divergence, and confirmed empirically that consecutive trade directions show a first-order negative correlation caused by the bid-ask bounce. Onofri et al. (2025) extended this framework into an "all-offset" aggregation construction and, on 8 US stocks and 1 ETF (Exchange-Traded Fund, a basket of assets traded as a single security on an exchange), observed that the trades per second and the required aggregation level ℓ^* move in the same direction. These two papers are the direct methodological predecessors of this work.

Gap. Combining the two lines, the gap can be summarised as two points:

- **(i) Cross-market transfer of method.** The randomness-testing framework based on D and

the all-offset construction has been validated on US stocks (Onofri et al., 2025; Shternshis and Marmi, 2025), but whether it also applies to the 24/7 continuously-trading cryptocurrency market, and what aggregation axis and level are needed there, has not been systematically studied.

- **(ii) End-to-end engineering pipeline.** Protocol-level financial beacons (Chiba and Ichikawa, 2024; Clark and Hengartner, 2010; Landis and Bonneau, 2025) and the underlying statistical validation (Onofri et al., 2025; Shternshis and Marmi, 2025) have not yet been connected on cryptocurrency data into a single end-to-end pipeline that runs from aggregation-parameter selection through to a password generator.

The thesis-level question of this work is: in a passive statistical setting, can public cryptocurrency market data, after statistical aggregation and cryptographic conditioning, provide usable entropy for strong password generation? This overall question is split below into three research questions: whether aggregation can extract statistically independent random sequences (RQ1), to what extent the resulting sequences pass randomness tests (RQ2), and whether, after cryptographic conditioning, they can support a password-generation application (RQ3).

This thesis addresses the question above through a staged empirical design.

1.3 Research Questions

Based on the motivation and the literature gaps above, this thesis answers the three research questions:

RQ1. Can aggregation extract statistically independent random sequences from cryptocurrency price data?

RQ2. To what extent can these sequences pass standard randomness tests?

RQ3. Can this data-driven randomness be effectively applied to secure password generation?

A note on the scope of RQ3: this thesis positions market-derived bits as entropy input to a standard password-conditioning pipeline. Statistical tests screen the quality of that input, while the final password output is produced by the conditioning and character-mapping procedure.

1.4 Thesis Structure

The remainder of this thesis is organised as follows.

- Chapter 1 introduces the motivation, the literature gaps, and the three research questions answered by this thesis.
- Chapter 2 gives the background needed for the later chapters, including prior work on financial data as a randomness source, market microstructure, randomness testing, and password-strength measures.
- Chapter 3 describes the pipeline of this thesis: how price data is turned into bit streams, how candidate sequences are screened, how several assets are combined, and how the strength of the final output is measured.
- Chapter 4 reports four experiments that move from the raw market data through the aggregated sequences to the multi-asset validation.

- Chapter 5 checks whether the validated sequences can drive a password-generator prototype.
- Chapter 6 answers the research questions and discusses the conditions and boundaries under which the results hold.
- Chapter 7 summarises the contributions of the thesis.

Chapter 2

Background

2.1 Literature Review

This section reviews papers related to the statistical properties and randomness of financial time series, following the two-line split introduced in the Brief Literature Review. The statistical validation line (Cont, 2001; Onofri et al., 2025; Shternshis and Marmi, 2025; Shternshis et al., 2022) focuses on the randomness structure of financial time series itself. The application / protocol line (Bonneau et al., 2015; Chiba and Ichikawa, 2024; Clark and Hengartner, 2010; Landis and Bonneau, 2025; Pierrot and Wesolowski, 2018) focuses on turning that data into cryptographic components.

Stylised facts and microstructure noise

On the statistical properties of high-frequency financial time series, Cont (2001) summarised a set of "stylised facts" from the returns viewpoint. These include heavy-tailed distributions, volatility clustering, slow power-law decay of the autocorrelation of absolute returns, and — most relevant to this thesis — negative autocorrelation at very short lags (microstructure scale) caused by the bid-ask bounce. That is, trade prices switch frequently between the best bid and the best ask, which makes the bit sequence encoded by the sign of Δp strongly negatively correlated at neighbouring positions. Bouchaud et al. (2002) described the same problem from the order-book side, where price changes are shaped by the limit order book and not by independent draws. Shternshis and Marmi (2025) formalised this negative autocorrelation as a testable first-order residual diagnostic, and provided empirical evidence on three low-price stocks: SNAP, F, and CCL.

In addition to the bid-ask bounce, another well-documented stylised fact at the microstructure level is the long memory of trade signs. Lillo and Farmer (2004) showed on the London Stock Exchange that the signs of executed market orders follow a long-memory process, with autocorrelation that decays slowly. They attribute this to order splitting and herding behaviour. The two stylised facts work in opposite directions on the lag-1 autocorrelation of price-change signs: the bid-ask bounce produces negative autocorrelation by alternating between the bid and the ask, while persistent trade-sign autocorrelation produces positive autocorrelation when a sustained order flow walks the book. Which mechanism dominates depends on the activity regime of the asset.

Together, these works show that any approach which directly treats the "sign of price change" as independent and identically distributed (i.i.d.) random bits is almost certain to fail at the raw tick level. This is the fundamental reason why this thesis treats "aggregation" as the core of the method.

Financial data as a randomness source

This line of work can be split into three complementary levels of abstraction.

1. **Protocol level.** Clark and Hengartner (2010) used computational finance tools to estimate that each stock in the Dow Jones 30 (a US index of 30 large-cap companies) provides 6–9 bits of entropy per day. Based on this, they designed a 128-bit verifiable beacon protocol. Blockchain data is a parallel route at the same level: Bonneau et al. (2015) studied Bitcoin block hashes as a public randomness beacon, and Pierrot and Wesolowski (2018) analysed how such entropy can be malleable under miner or adversarial influence. This thesis does not use blockchain data; it uses financial market data as the entropy input.

2. **Bit extraction level.** Chiba and Ichikawa (2024) used cryptocurrency prices to perturb the sampling interval of an LFSR, produced a bit stream, and ran Diehard tests (a classic randomness test battery, the historical precursor of NIST STS) as a sanity check — this answers how market signals can be engineered and whitened into usable random bits.

3. **Adversarial security level.** Landis and Bonneau (2025) showed under an active attacker model that manipulating an S&P 100 beacon (a US index of 100 large-cap companies) would require billions of dollars of capital and millions of dollars of slippage cost — this moves the question from "how much entropy is there" to "is the entropy trustworthy under adversarial conditions?".

These three levels answer different questions, but none of them connects all the way from "is the underlying data statistically usable" to an application-level password prototype. This is the second gap from the Brief Literature Review made concrete on cryptocurrency data. This thesis sits mainly at the bit extraction level, but differs from Chiba and Ichikawa (2024) in that, at the randomness-evaluation stage, it does not depend on extra LFSR post-processing. Instead, it directly evaluates whether the aggregated raw data itself satisfies modern randomness tests. It also does not enter the active-attacker level studied by Landis and Bonneau (2025); the scope is a passive statistical setting, which is sufficient for the password-generator evaluation in this thesis (see § 6.1).

Entropy-based predictability and the all-offset construction

Shternshis and Marmi (2025) proposed an entropy-based predictability test D based on the Kullback–Leibler divergence (formal definition in § 2.2). The test treats k as a user-defined hyper-parameter; Shternshis and Marmi (2025) use the fixed setting $k = 2$ in their §4.4 to study first-order pair-of-signs predictability (the bid-ask bounce mentioned above). This thesis adopts the same $k = 2$ setup as a first-order diagnostic, complementary to the adaptive-k main statistic, which gives the methodological starting point for separating first-order structure from higher-order structure. The D test is run at both adaptive k and fixed $k = 2$.

Onofri et al. (2025) further upgraded this framework into an "all-offset" construction (which Onofri *et al.* call "ℓ-samplings"): for each aggregation level ℓ, it constructs ℓ separate bit streams from ℓ different starting offsets, where each offset means starting at position o and taking every ℓ-th observation, for $o = 0, 1, \ldots, \ell - 1$. It then tests each stream independently and uses a pass-rate threshold as the acceptance rule for ℓ. This construction replaces "does a single p-value pass the line" with "do most offsets pass the line at the same time", and uses a box plot of $-\log_{10}(p)$ to directly show how the p-value distribution shifts as ℓ grows. The paper produces a robust empirical observation on 8 US stocks plus 1 ETF: the higher the trades per second of an asset, the larger the minimum required aggregation level ℓ^* (the original paper also notes that some assets show non-monotonic behaviour).

This thesis directly reuses this all-offset construction and pass-rate threshold framework, but differs from Onofri et al. (2025) in three concrete ways: the market is cryptocurrency rather than US equities; the aggregation axis includes physical time (1-second bars) in addition to transaction time; and the qualitative acceptance criterion of that work is operationalised as a quantitative fixed-threshold criterion together with a heuristic relaxed criterion (§ 3.4 and § 3.5).

Market efficiency under stress

Shternshis et al. (2022) used Shannon entropy, KL distance, and Monte Carlo simulation to show that the efficiency of the Moscow Stock Exchange in 2012–2021 changed significantly over time and depended on the industry sector. This is independent evidence from another type of market that "the properties of financial time series are not stationary". This observation supports reporting the selected ℓ^* across monthly time windows rather than treating it as a stable value.

2.2 Background Information

This section reviews the core concepts used in the later Methods chapter. All randomness tests are based on the same null hypothesis H_0: the sequence is i.i.d. Each test gives a p-value, and this thesis uses $\alpha = 0.01$ uniformly as the acceptance threshold.

Shannon entropy and conditional unpredictability. For a binary random variable $X \in \{0, 1\}$, the Shannon entropy is defined as $H(X) = -\sum p(x) \log_2 p(x)$ (Shannon, 1948), and reaches its maximum value of 1 when $p(0) = p(1) = 0.5$. It is worth emphasising that a marginal entropy close to 1 does not imply that the sequence is unpredictable — a sequence that simply alternates between 0 and 1 also has marginal distribution 0.5/0.5, but its conditional entropy $H(X_t \mid X_{t-1}) \approx 0$. The criterion for "is it random" used in this thesis is always based on conditional distributions or multi-symbol statistics, not on H alone. Throughout this thesis, the deviation $|H(X) - 1|$ is reported as the *Shannon bias*, used as a summary indicator only and not as an acceptance criterion. Mutual information, conditional entropy, and min-entropy are standard information-theoretic concepts; a general reference is Cover and Thomas (2006).

Predictability D. The entropy-based predictability test proposed by Shternshis and Marmi (2025) is based on the Kullback–Leibler divergence. It measures how much the joint distribution of consecutive symbol blocks of length k in the sequence deviates from the distribution that would be expected under i.i.d. Under H_0 it asymptotically follows a χ^2 distribution with $(s^{k-1} - 1)(s - 1)$ degrees of freedom (which equals $2^{k-1} - 1$ in the binary case). The window length k is a user-chosen hyper-parameter.

NIST SP800-22 STS. The NIST Statistical Test Suite (STS) is a randomness test battery published by NIST (Bassham et al., 2010). It contains 15 sub-tests in 5 categories: Frequency, Pattern, Entropy / Complexity, Spectral, and Random Walks. Common representative sub-tests include:

- **Monobit (frequency category).** It uses the standard-normal approximation of $|S_n|/\sqrt{n}$ to detect deviation of the frequency of 1s from 0.5.
- **Runs (pattern category).** It splits the sequence into maximal same-symbol segments and uses "the deviation of the number of segments from the i.i.d. expectation" as the statistic, which is sensitive to correlation between neighbouring bits.
- **Approximate Entropy (entropy / complexity category).** It measures the complexity of the sequence by taking the difference of the appearance frequencies of windows of length m

and $m+1$; the window length m is a user-chosen hyper-parameter. The original Approximate Entropy idea is due to Pincus (1991).

TestU01 batteries. TestU01 is a randomness test library (L'Ecuyer and Simard, 2007), and it has also been used by Onofri et al. (2025) on financial data. Per the TestU01 user guide, it contains sub-batteries such as Alphabit (9 sub-tests, friendly to small samples) and Rabbit (26 sub-tests, comprehensive). This thesis uses Alphabit in the main analysis as a cross-battery check complementing NIST STS; Rabbit is left outside the current scope and treated as future work.

Aggregated trades and 1-second bars. Binance provides two main data views: tick-by-tick trades (`trades`) and aggregated trades (`aggTrades`). The latter merges trades that happen at the same price, in the same direction, and at the same time into a single record. This reduces data size while keeping the price-change and sign information. Based on these two views, aggregation can be done on the transaction-time axis (by trade count) and on the physical-time axis (by second) respectively.

Multiple testing. The Bonferroni and Šidák corrections (Bonferroni, 1936; Šidák, 1967) control the family-wise Type-I error rate, by tightening the per-test significance level α to α/N or $1-(1-\alpha)^{1/N}$. They address the question "how often does *at least one* of N tests falsely reject H_0?". There is no symmetric correction for the opposite question, "how often does *at least one* of N tests falsely pass?", because tightening α makes each individual test more likely to pass and therefore pushes the family-wise pass probability in the wrong direction. False discovery rate control, such as the Benjamini–Hochberg procedure (Benjamini and Hochberg, 1995), is another possible approach, but this thesis does not use FDR correction; the reason is given in § 3.4.

Cryptographic conditioning. A standard cryptographic pipeline can take a rough entropy input, whose min-entropy may be below 1 bit per symbol, and condition it into pseudo-random output. This thesis uses **HKDF**, the HMAC-based Extract-and-Expand Key Derivation Function (Krawczyk, 2010; Krawczyk and Eronen, 2010), with **SHA-256** as the hash function (National Institute of Standards and Technology, 2015). HKDF has two steps: *Extract* compresses an entropy input, which may not be uniform, into a pseudo-random key; *Expand* then uses that key together with a context label to produce output bits of the required length. This combination works as a cryptographic conditioning / extraction component only when the IKM has enough min-entropy and the standard HKDF usage assumptions are met. This thesis follows the standard HKDF names:

- **salt** – a random or pseudo-random byte string given to the Extract step. Standard HKDF usage allows the salt to be public; however, if the HKDF output itself must remain secret, the conditioning stage must also include some non-public deployment-secret material. That secret may be carried by the salt, or it may enter the system design as a separate secret input.
- **IKM (input keying material)** – the entropy input to the Extract step, and the object sized by the min-entropy budget.
- **PRK (pseudo-random key)** – the 32-byte intermediate output:

$$\text{PRK} = \texttt{HKDF-Extract}(\text{salt}, \text{IKM}).$$

- **info** – a context label given to the Expand step.
- **OKM (output keying material)** – the variable-length byte output:

$$\text{OKM} = \texttt{HKDF-Expand}(\text{PRK}, \text{info}, L).$$

The min-entropy budget follows a simplified subset of NIST SP800-90B (Turan et al., 2018): this thesis uses the Most Common Value and Markov estimators, takes the smaller value, and does not

claim to implement the full SP800-90B entropy validation flow. The estimator output is used in § 3.7 and § 4.5.5.

Password output strength. Randomly generated passwords and human-chosen passwords should not be measured in the same way. For a uniformly generated password, the main strength measure is search-space entropy: a password of length L over a character set of size n has $L \log_2 n$ bits of search space. Human-chosen passwords need guessability models, because people reuse words, dates, keyboard patterns, and other regular structures. Large empirical studies place many human-chosen passwords in the tens-of-bits range (Bonneau, 2012; Florêncio and Herley, 2007). Modern verifier guidance also focuses on length, blocklists, rate limiting, and secure verifier-side storage, rather than fixed composition rules (National Institute of Standards and Technology, 2025).

This thesis later adopts $\gtrsim 80$ bits as a working benchmark for a strong password. The value is used here as an engineering reference point on the search-space-entropy scale, and it is close to the 75–80 bit level that recurs in industry practitioner guidance: Proton describes 75–80 bits as the level that resists currently known brute-force attacks (Proton, 2023), Keeper Security recommends at least 75 bits (Keeper Security, 2024), the KeePass documentation uses 80 bits as the example minimum quality for a master password (KeePass, n.d.), and the EFF six-word Diceware passphrase corresponds to about 77 bits (Electronic Frontier Foundation, 2016). This thesis takes $\gtrsim 80$ bits because it is well above the $\sim$ 20–30 bit range of human-chosen passwords and because the 2^{80} search space is well outside the practical reach of known offline cracking.

Chapter 3

Methods

This chapter describes all the methods used in this thesis to turn a price sequence into a random bit stream that can be tested. It is organised in the order "encoding → construction → tests → acceptance rule → axis comparison → multi-asset combination → output metrics (throughput and min-entropy)."

- **Reused from prior work.** The entropy-based predictability test D from Shternshis and Marmi (2025), run at both adaptive k and fixed $k = 2$; the all-offset construction from Onofri et al. (2025), including its choice of $m = 5$ for Approximate Entropy; and the classical sub-tests from the NIST SP800-22 STS battery (Bassham et al., 2010) (Monobit, Runs, Approximate Entropy, and the extended-battery items) and TestU01 Alphabit (L'Ecuyer and Simard, 2007). The strict criterion (§ 3.4) operationalises the all-offset framework of Onofri et al. (2025) with a fixed 0.80 pass-rate threshold.
- **New in this thesis.** The overall methodological idea is to place the entropy-based D test and the classical NIST/TestU01 sub-tests under one all-offset acceptance and audit framework, and to evaluate the resulting combined battery on the 24/7 continuous cryptocurrency market. On top of the reused components, the concrete new contributions are: the relaxed criterion (§ 3.4), the 1-second bar physical-time pipeline (§ 3.5), the multi-asset XOR combination (§ 3.6), and the output metrics for throughput and a simplified min-entropy estimate (§ 3.7). HKDF-SHA256 is adopted as a standard cryptographic conditioning component (introduced in § 2.2); it serves the application in Chapter 5 and is not itself counted among the new contributions of this thesis.

3.1 Aggregation and Encoding

Let the original tick sequence be $\{(t_i, p_i)\}_{i=0}^{N-1}$, where t_i is the timestamp and p_i is the trade price (0-based indexing, consistent with the code implementation). An aggregation level ℓ and a starting offset $o \in \{0, 1, \ldots, \ell - 1\}$ together define a sampled subsequence

$$q_j^{(\ell,o)} = p_{o+j\ell}, \qquad j = 0, 1, \ldots, \lfloor (N - o - 1)/\ell \rfloor. \tag{3.1}$$

Its difference is $\Delta q_j^{(\ell,o)} = q_j^{(\ell,o)} - q_{j-1}^{(\ell,o)}$, for $j = 1, 2, \ldots, \lfloor (N - o - 1)/\ell \rfloor$. The bit encoding takes the sign:

$$b_j^{(\ell,o)} = \mathbb{1}\left[\Delta q_j^{(\ell,o)} > 0\right]. \tag{3.2}$$

If $\Delta q_j^{(\ell,o)} = 0$, that bit is removed (*drop-zero*, consistent with Shternshis and Marmi (2025)) and the later bits shift forward to keep the bit stream continuous. This removal is necessary: the share of zero differences is large at small ℓ, and mapping zeros to either 0 or 1 by any fixed rule would inject a clear bias.

On the transaction-time axis, the unit of ℓ is "number of trades". On the physical-time axis, p_i is first converted into "per-second close price" through the 1-second bar pipeline described in § 3.5, and the unit of ℓ then becomes "number of seconds". All other encoding steps are the same on the two axes, which allows the two axes to be compared under the same encoding logic.

Algorithm 1 Aggregation and bit encoding (single offset)

Require: Tick prices $\{p_0, p_1, \ldots, p_{N-1}\}$, level ℓ, offset o
Ensure: Bit stream b

$q \leftarrow [\, p_o,\ p_{o+\ell},\ p_{o+2\ell},\ \ldots\,]$
$b \leftarrow [\,]$
for $j = 1$ **to** $|q| - 1$ **do**
    $\Delta \leftarrow q[j] - q[j-1]$
    if $\Delta > 0$ **then**
        append 1 to b
    else if $\Delta < 0$ **then**
        append 0 to b
    else
        skip ▷ drop-zero
    end if
end for
return b

3.2 All-Offset Construction

The all-offset construction described in this section comes directly from the "ℓ-samplings" framework proposed by Onofri et al. (2025) — for each aggregation level ℓ, ℓ separate bit streams are built, one from each starting offset ($o = 0, 1, \ldots, \ell - 1$). This thesis follows this construction, and only adjusts the parameters on the cryptocurrency data (per-asset ℓ grid, step, and upper bound).

For a fixed ℓ, there are ℓ candidate offsets. The construction considers all ℓ bit streams together. Let $p^{(o)}$ denote the test p-value on each stream; the pass rate is defined as

$$r_{\text{pass}}(\ell, \alpha) = \frac{1}{N_{\text{valid}}(\ell)} \sum_{o \in \mathcal{O}_{\text{valid}}(\ell)} \mathbb{1}\left[p^{(o)} \geq \alpha\right], \tag{3.3}$$

where $\mathcal{O}_{\text{valid}}(\ell)$ is the set of offsets with bit count at least `MIN_BIT_COUNT` $= 2000$. This threshold is a conservative engineering floor that ensures enough samples to support the χ^2 asymptotic approximation of the D test. Because different offsets share the same underlying tick data, the ℓ bit streams are strongly correlated (especially for large ℓ); their simultaneous passing provides offset-robustness evidence. This point determines the design choice for the acceptance rule in § 3.4.

3.3 Test Battery

The test battery of this thesis is as follows:

- **Predictability D (adaptive k).** The main statistic, with $k = \lfloor 0.5 \log_2 n \rfloor$.
- **Predictability D (fixed $k = 2$).** A first-order diagnostic for pair-of-signs dependence between neighbouring symbols, covering both the bid-ask-bounce reversal and the persistent same-direction order-flow stylised facts.
- **Monobit.** A basic frequency test that checks the overall 0/1 balance.
- **Runs.** NIST-style, checking correlation between neighbouring bits. The NIST pre-frequency check is not implemented separately, since Monobit passing at $\alpha = 0.01$ implies $|\pi - 0.5| \leq 1.288/\sqrt{n}$, which is strictly tighter than the NIST threshold $2/\sqrt{n}$; the check is therefore equivalently absorbed by the Monobit criterion on every accepted ℓ.
- **Approximate Entropy ($m = 5$).** The multi-window complexity difference (Pincus, 1991), aligned with Table 4 of Onofri et al. (2025). The NIST default is $m = 2$ (more stable on short sequences); $m = 5$ is more sensitive to short-range structure but requires $n \gtrsim 10 \cdot 2^m \approx 320$ for the asymptotic distribution to be reliable. This thesis uses $n \geq 2000$, well above this threshold.
- **TestU01.** TestU01 (L'Ecuyer and Simard, 2007) is used for cross-battery external validation in the extended-audit and validation stages. Alphabit is enabled; Rabbit is reserved for future work.

Under the extended-battery convention, this thesis uses a fixed **29-sub-test universe**: 5 core tests (D at adaptive k, $D(k = 2)$, Monobit, Runs, and ApEn), 7 NIST SP800-22 extensions (Bassham et al., 2010) (Block Frequency, Cumulative Sums forward/backward, Longest Run, DFT, and Serial $m/m{-}1$), and 17 TestU01 Alphabit statistics. Alphabit has 9 named tests, but RandomWalk1 outputs 5 statistics at each of L64 and L320; this thesis therefore counts Alphabit at the statistic level, giving 17 sub-tests. The three parts sum to $5 + 7 + 17 = 29$. At short lengths, TestU01 may length-skip long-block sub-tests, and such entries are excluded from the admissible denominator.

Sub-test selection under a length budget. The bit-stream lengths the cells in this thesis can reach are roughly in the 2K–200K bit range (single-offset streams are about 2K–25K bit; cross-offset, cross-month, or cross-asset concatenations are about 25K–200K bit). Several sub-tests in the full NIST SP800-22 STS battery (15 items) and the TestU01 Crush battery recommend input lengths on the order of 10^6 bits – for example Non-overlapping Template Matching, Universal Statistical, the Random Excursions family, and most of Crush – which exceeds the cell-reachable range and cannot run stably at the selected ℓ^*. This thesis therefore selects from NIST SP800-22 only the 7 sub-tests whose length requirement is at most $\approx$ 200K bit (Block Frequency, Cumulative Sums forward/backward, Longest Run, DFT, Serial $m/m{-}1$), and from TestU01 only the Alphabit sub-battery. These 7 NIST SP800-22 sub-tests span the main NIST STS families – frequency (Block Frequency), cumulative offset (Cumulative Sums), runs (Longest Run), spectral (DFT), and entropy / multi-symbol structure (Serial) – so that the length-budget filter still retains family-level coverage. The omitted items remain methodologically relevant and are listed under § 6.4.

When TestU01 is enabled. Following the cross-battery design of Onofri et al. (2025), the methodology of this thesis further includes the Alphabit sub-battery of TestU01 as cross-battery external validation. TestU01 is not enabled during the selection-grid stage: the per-(asset, ℓ, offset) bit stream is only an intermediate output of parameter search, and its length is usually below the range where the cross-battery sanity check applies, so it is not a suitable input for cross-battery validation. The

extended-audit and validation stages concatenate across months, assets, or offsets at the selected ℓ^*, so that the length meets the sanity-check range, and the corresponding Alphabit sub-tests are then enabled. Rabbit requires longer and more stable bit streams and is therefore not part of the main analysis.

Why both fixed $k = 2$ and adaptive k? Adaptive $k = \lfloor 0.5 \log_2 n \rfloor$ is usually 5 to 6 on ultra-high-frequency monthly samples. The D test then has $2^{k-1} \approx 16$ to 32 contexts; first-order correlation – whether the symbol reversal caused by the bid-ask bounce, or the same-direction persistence caused by sustained order flow — is diluted across more than ten or even tens of positions, and the statistical power drops quickly. Shternshis and Marmi (2025) provides empirical evidence consistent with this: on predictable days, with fixed $k = 2$, the "too strong sign reversal" pattern (i.e., $\hat{p}(00)+\hat{p}(11) < 0.5$) is significant; when k is raised to about 5 to 6, this pattern is no longer significant. Therefore, $D(k = 2)$ is kept as a first-order diagnostic for pair-of-signs dependence, complementary to $D(\text{adaptive } k)$. The former captures short-range structure, while the latter measures the overall predictability. The later baseline results further verify that first-order residuals in cryptocurrency tick signs need to be diagnosed explicitly, rather than being left to the adaptive-k main statistic alone.

3.4 Acceptance Criteria

The acceptance rule is the key step in the all-offset framework that maps the "pass rate" to "whether ℓ is accepted". This thesis uses two criteria, introduced in chronological order.

Strict criterion. Onofri et al. (2025) apply a qualitative acceptance criterion. This thesis adopts a quantitative counterpart within the same framework: a fixed 0.80 pass-rate threshold, used consistently across all (asset, period) cells reported in this thesis. The strict criterion requires the following three conditions to hold simultaneously:

$$\begin{aligned} r_{\text{valid}}(\ell) &\geq 0.80 \\ r^{D}_{\text{pass}}(\ell) &\geq 0.80 \\ r^{\text{Mono}}_{\text{pass}}(\ell) &\geq 0.80 \end{aligned} \tag{3.4}$$

Here $\alpha = 0.01$ is taken from Onofri et al. (2025). The pass rates of Runs and $D(k = 2)$ are recorded as supplementary diagnostics only and do not enter the acceptance criterion; the reason is given in § 3.3.

Relaxed criterion (a heuristic newly introduced in this thesis). The all-offset bit streams are strongly correlated. From an engineering point of view, the final system only needs to pick one usable selected output offset; but in the statistical selection process, a simple "at-least-one passes" rule would introduce a serious selection bias. If the underlying sequence still contains structural dependence, a large number of offsets may still produce one offset that passes the α threshold because of sample variation or low test power.

As discussed in § 2.2, Bonferroni / Šidák corrections address the wrong direction for this "false pass" question. This thesis therefore neither adopts the "at-least-one passes" rule (selection bias) nor adopts the Bonferroni / Šidák formal correction (the direction is reversed), and instead uses the following heuristic redundancy criterion:

$$\text{accept}_{\text{relaxed}}(\ell) \iff n_{\text{pass}}(\ell) \geq \max(F,\ \lceil f \cdot N_{\text{valid}}(\ell) \rceil)\,, \tag{3.5}$$

where $n_{\text{pass}}(\ell)$ is the number of offsets that pass both D and Monobit (at $\alpha = 0.01$, uncorrected);

(F, f) are two tuning parameters. This thesis takes $(F, f) = (3, 0.03)$: F gives the minimum redundancy of joint confirmation across multiple offsets, and f is about three times the $\alpha = 0.01$ noise floor. This rule is explicitly labelled in the thesis as a *heuristic robustness check*, not a formal correction.

Algorithm 2 All-offset acceptance check at level ℓ

Require: Tick prices $\{p_i\}$, level ℓ, criterion $\in \{\text{strict}, \text{relaxed}\}$, $\alpha = 0.01$
Ensure: accept$(\ell) \in \{\text{true}, \text{false}\}$; selected output offset (if accepted)

$\mathcal{O}_{\text{valid}} \leftarrow \emptyset$
for $o = 0$ **to** $\ell - 1$ **do**
  $b \leftarrow$ Algorithm 1 with (p, ℓ, o)
  if $|b| \geq$ `MIN_BIT_COUNT` $(= 2000)$ **then**
    compute and store $p^D(o),\ p^{\text{Mono}}(o)$ on b ▷ reused by both criteria below
    add o to $\mathcal{O}_{\text{valid}}$
  end if
end for
$N_{\text{valid}} \leftarrow |\mathcal{O}_{\text{valid}}|$
$r_{\text{valid}} \leftarrow N_{\text{valid}}/\ell$
if criterion $=$ strict **then**
  $r^D_{\text{pass}} \leftarrow |\{o \in \mathcal{O}_{\text{valid}} : p^D(o) \geq \alpha\}|/N_{\text{valid}}$
  $r^{\text{Mono}}_{\text{pass}} \leftarrow |\{o \in \mathcal{O}_{\text{valid}} : p^{\text{Mono}}(o) \geq \alpha\}|/N_{\text{valid}}$
  return $r_{\text{valid}} \geq 0.80 \ \wedge\ r^D_{\text{pass}} \geq 0.80 \ \wedge\ r^{\text{Mono}}_{\text{pass}} \geq 0.80$
end if
if criterion $=$ relaxed **then**
  $n_{\text{pass}} \leftarrow |\{o \in \mathcal{O}_{\text{valid}} : p^D(o) \geq \alpha \ \wedge\ p^{\text{Mono}}(o) \geq \alpha\}|$
  return $n_{\text{pass}} \geq \max(F,\ \lceil f \cdot N_{\text{valid}} \rceil)$ ▷ $(F, f) = (3, 0.03)$
end if

Selection rule for ℓ^*. The smallest ℓ for which Algorithm 2 returns true is recorded as the selected ℓ^*. The specific ℓ grid (per-asset values, step, and bounds) is given in Chapter 4.

3.5 Time-Based Aggregation Pipeline

Adaptations for the cryptocurrency market. To take advantage of the 24/7 continuous nature of the cryptocurrency market, this thesis introduces a physical-time aggregation axis based on 1-second bars, in addition to the transaction-time axis used above. On the transaction-time axis the throughput moves with the trading frequency, so the time needed to produce one bit varies with how active the market is. On the physical-time axis, $1/\ell^*$ gives a fixed sampling-rate and a simple wall-clock waiting-time view. However, $1/\ell^*$ is not the final effective bit rate: because the drop-zero step removes seconds with no price change, the actual bit/s is usually slightly lower.

The 1-second bar pipeline (Algorithm 3) groups trades into one-second buckets indexed by UTC time (Coordinated Universal Time, the global timestamp standard used by Binance). For each second that contains no trade, the bucket is filled with the price of the most recent prior trade (forward-fill). The standard difference + sign + drop-zero encoding from § 3.1 is then run on the resulting per-second close-price sequence. Forward-filled seconds have zero difference with their predecessor and are dropped at the drop-zero step, so they contribute no bit; the bit stream therefore contains only bits derived from real price changes between actually-traded seconds.

Algorithm 3 1-second bar pipeline (physical-time axis)

Require: Trade stream $\{(t_i, p_i)\}$, level ℓ (seconds)
Ensure: Bit streams $\{b^{(\ell,o)}\}$ for $o = 0, \ldots, \ell - 1$

▷ Step 1: bucket by UTC second + forward-fill empty seconds
▷ (the first observed second always contains a trade, since data starts at the first trade)

group trades by $\lfloor t_i \rfloor$ ▷ to whole UTC seconds
for each second s in order **do**
  if s has trade(s) **then**
    $c_s \leftarrow$ price of last trade in s
  else
    $c_s \leftarrow c_{s-1}$ ▷ forward-fill
  end if
end for

▷ Step 2 + 3: encode each offset via Algorithm 1

for $o = 0$ **to** $\ell - 1$ **do**
  $b^{(\ell,o)} \leftarrow$ Algorithm 1 with $(\{c_s\}, \ell, o)$
end for
return $\{b^{(\ell,o)}\}$

The ℓ grid is taken as $[10, 600]$ with step 1. This covers both the optimistic case where "second-level aggregation is enough" and a longer (minute-level) aggregation for comparison. The upper bound of 600 seconds (=10 minutes) corresponds to a reasonable latency upper bound for password generation.

3.6 Multi-Asset XOR Combination

This thesis also defines a multi-asset combination operator on top of the single-asset physical-time bit streams. Given an asset subset S, for each asset $a \in S$ the per-second close-price sequence is first built (as produced by the 1-second bar pipeline of § 3.5) and the sign is computed as

$$s_{a,t} = \operatorname{sign}(c_{a,t} - c_{a,t-1}) \in \{-1, 0, +1\}. \tag{3.6}$$

The value 0 here means the second has the same price as the previous second (including seconds where forward-fill leaves no real price change). Unlike single-asset encoding, multi-asset combination cannot apply drop-zero to each asset independently first, because the per-asset bit streams would then have different lengths and the UTC-second alignment would be lost. This thesis therefore first aligns the asset subset on UTC seconds and then applies a *drop-any-zero* rule: a second enters the combined stream only when every $a \in S$ satisfies $s_{a,t} \neq 0$. Drop-any-zero is the multi-asset generalisation of the drop-zero rule in § 3.1; it reduces to ordinary drop-zero when $|S| = 1$.

The remaining seconds t are encoded as

$$x_{S,t} = \bigoplus_{a \in S} \mathbb{1}[s_{a,t} > 0], \tag{3.7}$$

where $\oplus$ denotes XOR. The resulting $\{x_{S,t}\}$ is called the *combined stream*. An ℓ-level XOR aggregation is then applied to the combined stream: for each offset $o \in \{0, \ldots, \ell - 1\}$, consecutive blocks

of ℓ combined bits are XOR-reduced,

$$y_{S,j}^{(\ell,o)} = \bigoplus_{r=0}^{\ell-1} x_{S,\, o+j\ell+r}. \tag{3.8}$$

The use of XOR here comes from the piling-up result in linear cryptanalysis (Matsui, 1994). For a binary stream, this thesis understands the *marginal bias* as how far the single-bit distribution departs from uniform, that is, how far $p(1)$ deviates from 0.5. In the ideal case of independent inputs, XOR-ing several slightly biased bits makes the output bias decay as the product of the input biases, so it moves closer to 0.5. This is a computable result that holds under independent inputs. Because the asset sign streams are correlated, piling-up is used here as an engineering assumption: if the assets still retain some independent component, the multi-asset XOR combination may weaken the single-asset residual. This assumption is then checked through the mutual-information diagnostic and out-of-sample validation.

Algorithm 4 Multi-asset XOR combination and ℓ-level aggregation

Require: Per-asset 1-second close prices $\{c_{a,t}\}$ for asset subset S, level ℓ, offset o
Ensure: Aggregated combined stream y

▷ Step 1: per-asset signs with zeros preserved

for each asset $a \in S$ **do**
  for each UTC second t **do**
    $s_{a,t} \leftarrow \mathrm{sign}(c_{a,t} - c_{a,t-1})$ ▷ -1, 0, or $+1$
  end for
end for

▷ Step 2: align and drop-any-zero

$T \leftarrow$ common UTC-second range covered by all assets in S
$x \leftarrow [\,]$
for each $t \in T$ **do**
  if $s_{a,t} = 0$ for any $a \in S$ **then**
    skip
  else
    append $\bigoplus_{a \in S} \mathbb{1}[s_{a,t} > 0]$ to x
  end if
end for

▷ Step 3: ℓ-level XOR aggregation on the combined stream

$y \leftarrow [\,]$
$j \leftarrow 0$
while $o + (j+1)\ell \le |x|$ **do**
  append $\bigoplus_{r=0}^{\ell-1} x[o + j\ell + r]$ to y
  $j \leftarrow j + 1$
end while
return y

This thesis refers to $\{x_{S,t}\}$ as the *combined stream*. The code base and some file names use the earlier term `fused_stream.bin`; the two refer to the same multi-asset XOR output.

The symbol ℓ has two distinct roles in this thesis. In § 3.1 (Algorithm 1), ℓ acts in the *price domain*: every ℓ-th price is subsampled, adjacent subsampled prices are differenced, and the difference is sign-encoded; this is equivalent to summing the ℓ tick-level price increments inside the window in real

arithmetic and then sign-encoding the sum into one bit (linear sum + sign). In Algorithm 4 above, the ℓ-level aggregation acts in the *bit domain*: the multi-asset combined bits are produced first, and ℓ consecutive $\{0, 1\}$ bits are then XOR-reduced over $\mathrm{GF}(2)$. The two operations share the symbol ℓ but differ in pipeline position and algebraic structure: the single-asset pipelines use the former (price-domain aggregation), while the multi-asset combined pipeline uses the latter (bit-domain XOR aggregation).

3.7 Output Metrics

This section defines two metrics used to evaluate the output bit streams produced by the methods above: a *throughput* metric for bit-production rate and waiting-time discussion, and a simplified *min-entropy budget* for sizing the input keying material of the downstream cryptographic conditioning component.

Throughput. To align the methodological results with the downstream application, this thesis records the throughput r_b at each accepted ℓ^*, defined as the effective output rate (in bits/s) of the selected output offset bit stream at that ℓ^*. On the transaction-time axis, r_b is a realised rate driven by market activity, so it changes with the asset and the monthly trades/s. On the physical-time axis, $1/\ell^*$ gives a fixed sampling-rate and an approximate waiting time per candidate bit. The effective r_b is usually slightly lower than $1/\ell^*$, because drop-zero removes seconds that do not produce a bit.

Min-entropy budget. This thesis uses a simplified min-entropy estimate to check whether a produced bit stream carries enough randomness to drive a standard cryptographic conditioning component. For each stream, the estimate is the smaller of two NIST SP800-90B non-IID estimators (Turan et al., 2018): Most Common Value (MCV), which measures the imbalance between 0 and 1, and the Markov estimator, which measures the dependence between adjacent bits. These two estimators are chosen because they cover the two most common sources of bias in a binary stream: single-bit imbalance and pairwise correlation. The result is reported as a per-symbol lower bound h_∞ and is used to determine the IKM size for downstream conditioning. For a binary stream, h_∞ has a theoretical maximum of 1 bit per symbol, which is reached only by a perfectly independent and balanced bit stream; this thesis treats "h_∞ close to 1" as the practical judgment direction for stream quality. If the deployment requires T bits of target entropy, the input size needed is $\lceil T/(h_\infty \times 8) \rceil$ bytes – $h_\infty = 1$ gives the lower bound $\lceil T/8 \rceil$ bytes, and the further h_∞ is from 1 the more input bytes are needed. This is not a complete SP800-90B non-IID certification (the full procedure includes more estimators); it is the simplified version used in this thesis.

Chapter 4

Experiments

This chapter first describes the experimental setup and data in the Overview (§ 4.1), then reports the results of Experiment 1 baseline (§ 4.2), Experiment 2 (§ 4.3), and Experiments 3–4 (§ 4.4, § 4.5) in turn. The Password Generator Prototype, as the application-layer implementation, is treated separately in Chapter 5; this chapter only covers the statistical experiment chain.

4.1 Overview

Experiments. Each experiment in this chapter answers one or more of the research questions in § 1.3.

- Experiment 1 (§ 4.2) answers the negative side of RQ1: are the raw tick sequences themselves random enough? The answer is no, and this directly motivates Experiment 2.
- Experiment 2 (§ 4.3) answers RQ1 and RQ2: at what aggregation level ℓ does the bit stream start to pass standard randomness tests, and how do the transaction-time and physical-time axes differ?
- Experiments 3 and 4 (§ 4.4, § 4.5) together with the password generator prototype in Chapter 5 answer RQ3: can these validated market-derived streams support secure password generation?

Common parameters. The same core parameters are used across all experiments: the significance level is $\alpha = 0.01$, `MIN_BIT_COUNT` is 2000, and the pass-rate threshold is 0.80. All runner scripts are written in Python with NumPy. The numbers reported in this chapter can be reproduced by running the `scripts/runner_*.py` files in the project repository.

Source. All data come from Binance's public spot `aggTrades` API. Monthly archive zip files are used instead of live-stream scraping, so that the data is complete. Each `aggTrades` record contains a timestamp, a price, a volume, and a buy/sell flag. This thesis uses only the timestamp and the price.

Asset universe. Five high-liquidity USDT (Tether, a US-dollar-pegged stablecoin) spot pairs on Binance: `BTCUSDT`, `ETHUSDT`, `BNBUSDT`, `SOLUSDT`, and `DOGEUSDT`. These five assets span a wide range of liquidity and microstructure regimes (Table 4.1), allowing the method's behaviour to be checked across different trades/s levels.

Period. The sample covers 15 months, from January 2025 to March 2026. Both Experiment 1 and Experiment 2 use one-month windows across all 15 months, so the analyses share a common (asset, month) cell. Table 4.1 below summarises the activity level of each asset across this sample period.

For each asset, the table reports the median and the [5th, 95th] percentiles of two rates: the raw trades per second (one record for each raw execution) and the aggTrades per second (the rate at which the encoding pipeline of § 3.1 actually emits an event).

Table 4.1: Per-asset trades/s for the five USDT spot pairs.

	Raw trades/s		aggTrades/s		raw/agg
Asset	median	[p5, p95]	median	[p5, p95]	median
BTCUSDT	50.8	[25.1, 68.6]	14.3	[8.7, 21.1]	3.3
ETHUSDT	43.9	[31.0, 70.9]	13.7	[10.9, 19.0]	2.9
SOLUSDT	25.0	[14.2, 36.8]	4.1	[2.9, 7.5]	5.4
DOGEUSDT	15.3	[9.0, 24.5]	3.6	[1.5, 7.1]	4.2
BNBUSDT	12.6	[8.4, 25.7]	4.3	[2.9, 8.4]	2.8

Throughout this thesis "/s" is shorthand for "per second". For each of the two rates (raw trades/s and aggTrades/s), the table reports the per-asset median and the [5th, 95th] percentiles across the 15 monthly archives covering the 2025-01 to 2026-03 sample period. Assets are sorted by the median raw trades/s in descending order.

aggTrades vs raw trades. The aggTrades/s rate counts records in Binance's public aggregated stream, which merges raw fills that share timestamp, price, and direction into a single record. The raw trades/s rate is recovered from the `aggTrades` side by summing $(\texttt{last\_trade\_id} - \texttt{first\_trade\_id} + 1)$ over all rows; downloading from the raw `trades` endpoint is therefore unnecessary. This raw count matches the granularity of the LOBSTER message-file counts used by Shternshis and Marmi (2025) and Onofri et al. (2025). Here, a taker order is the incoming order that removes liquidity, and maker fills are executions against resting orders. The raw/agg ratio is much higher on `SOLUSDT` and `DOGEUSDT` than on the other three assets, which suggests that on these two assets a single taker order tends to consume more maker fills before the price moves to the next level. This is consistent with their lower price levels and finer ticks compared to the typical trade size.

Pre-processing. On the transaction-time axis, the monthly `aggTrades` are sorted by time and rows with zero price difference are dropped. On the physical-time axis, the trades are sorted, grouped into UTC-second buckets, empty seconds are forward-filled with the previous price, and seconds with zero price difference are then dropped. Dropping the zero-delta items is necessary on both axes, because mapping a zero to either bit (0 or 1) by any fixed rule would inject a clear bias (§ 3.1).

4.2 Experiment 1: Baseline

Experiment 1 answers the negative side of RQ1. The simplest possible baseline is tested: treating the sign-encoded raw tick sequence directly as a random bit stream. If it passed, the aggregation studied in later sections would be unnecessary; since it does not, the result gives a quantitative reason to aggregate. This section reports the latter.

4.2.1 Setup

- **Configuration.** A special case of Algorithm 1 with $\ell = 1$ and $o = 0$: sort `aggTrades` by time, take the price difference, drop the zero-difference rows, and encode the sign of each remaining difference as one bit. There is no aggregation and no post-processing.

- **Asset universe.** The five USDT spot pairs used throughout this thesis (`BTCUSDT`, `ETHUSDT`, `BNBUSDT`, `SOLUSDT`, `DOGEUSDT`), spanning the full range of trades/s in Table 4.1. The five-asset sample is what makes the cross-asset patterns reported below visible: a single-asset baseline cannot show the activity-dependent differences that the rest of this chapter builds on.
- **Window.** The full 15-month sample, 2025-01-01 to 2026-03-31. The diagnostic granularity is monthly: one bit stream per (asset, month) cell, $5 \times 15 = 75$ cells in total.
- **Diagnostics.** For each monthly bit stream, the recorded quantities are: bit count n, bits per second, $p(1)$, Shannon bias $|H-1|$, Monobit p-value, Runs p-value, lag-1 autocorrelation ρ_1, the longest 0-run and 1-run lengths $L_{\max}$, and two structure tests — Approximate Entropy and the entropy-predictability test D at adaptive k and at fixed $k = 2$; the definition, parameters, and provenance of each test are given in § 3.3. The bit count n and bits per second are descriptive metadata. Lag-1 autocorrelation and the longest run are added as baseline diagnostics: the former measures short-range dependence directly, and the latter gives an intuition for long-structure cryptographic risk. ApEn and D test conditional structure, so the baseline is not judged only from marginal entropy or the 0/1 frequency.

4.2.2 Results

Table 4.2: Experiment 1 baseline — per-asset summary.

Asset	bps med.	$1 - H$ max	ρ_1 med.	Mono. rej.	Runs rej.	ApEn rej.	D rej.	D_2 rej.	$L_{\max}$ max
BTC	10.68	1.94×10^{-3}	+0.636	15/15	15/15	15/15	15/15	15/15	12,039
ETH	9.56	7.93×10^{-3}	+0.745	13/15	15/15	15/15	15/15	15/15	15,899
BNB	2.49	9.98×10^{-5}	+0.366	13/15	15/15	15/15	15/15	15/15	2,069
DOGE	2.02	1.35×10^{-5}	+0.477	8/15	15/15	15/15	15/15	15/15	5,071
SOL	1.74	2.35×10^{-6}	+0.095	4/15	15/15	15/15	15/15	15/15	792

Each row aggregates 15 (asset, month) cells covering 2025-01 to 2026-03: the median bit emission rate (bps), the worst-case Shannon bias (largest $1 - H$), the median lag-1 autocorrelation, the number of months (out of fifteen) on which Monobit, Runs, ApEn, D (adaptive k), and $D_2 = D(k{=}2)$ reject at $\alpha = 0.01$, and the largest 0-run or 1-run length $L_{\max}$ observed across all 15 cells. Rows are sorted by bps median in descending order. The adaptive block length $k = \lfloor 0.5 \log_2 n \rfloor$ falls in 10–12 on this sample. The per-(asset, month) distributions behind these summary statistics are shown in Fig. 4.1.

The marginal distribution is nearly balanced across all 75 baseline cells, with $1 - H$ at most 7.93×10^{-3} and as low as $\sim 10^{-5}$ on the lower-activity assets. However, the Shannon bias only describes the 0/1 ratio; it cannot test adjacent dependence or longer structure, so it is reported only as a summary indicator and does not enter the acceptance criterion. The five observations below are organised as a chain of evidence.

Conditional-predictability tests and Runs reject every cell. The entropy-predictability test D (adaptive k and fixed $k = 2$) tests whether the next bit depends on previous bits; Approximate Entropy measures the complexity of short patterns; the Runs test examines the run structure of adjacent bits. All four tests reject all 75 cells at $\alpha = 0.01$ — 15 out of 15 months for every asset (Table 4.2), with the Runs p-value underflowing to 0 on 73 of them. Unlike Monobit, these tests do not rely only on the marginal frequency moving away from 0.5: even for SOL, where $p(1)$ stays close to 0.5 and Monobit rejects only 4 of 15 months, D, ApEn, and Runs reject all 15 months. This is the most direct evidence that the raw tick stream is not random, and the evidence that depends least on any marginal assumption.

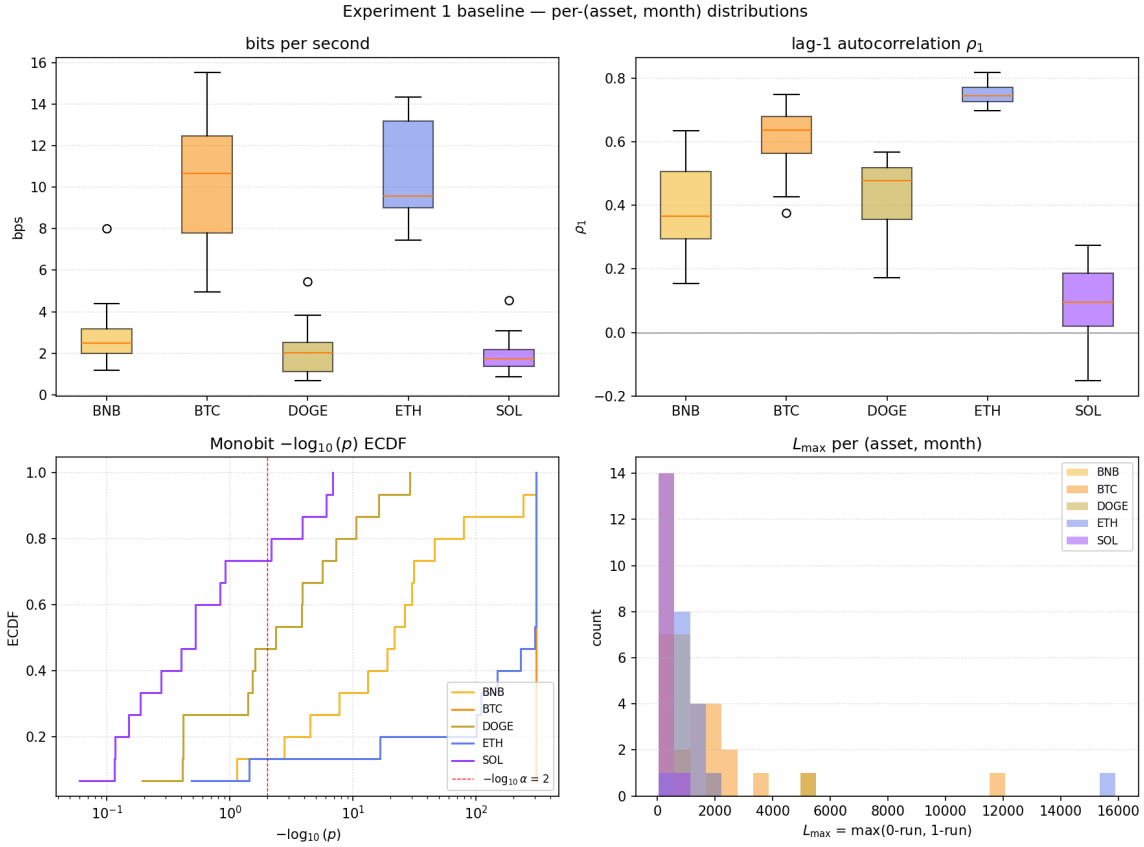

Figure 4.1: Experiment 1 baseline — per-(asset, month) distributions.

One observation per (asset, month) cell across the 15-month sample. *Top-left:* bits per second. *Top-right:* lag-1 autocorrelation ρ_1, with the zero line marked. *Bottom-left:* ECDF of $-\log_{10}(p_{\text{Monobit}})$, with the $\alpha = 0.01$ rejection threshold marked. *Bottom-right:* $L_{\max} = \max(0\text{-run}, 1\text{-run})$ histogram.

Monobit alone misses non-randomness. Of the 75 cells, 22 are not rejected by Monobit at $\alpha = 0.01$: 11 on SOL, 7 on DOGE, 2 each on BNB and ETH, and 0 on BTC. As the previous observation shows, these cells do not pass the randomness checks at the conditional-predictability or run-structure level. The non-rejections concentrate on the low-bps assets, because their $p(1)$ is closer to 0.5 and their shorter monthly bit streams give Monobit less power to detect a small marginal deviation. The apparent "Monobit reject count rises with bps" trend should therefore not be read as evidence that the low-bps assets are more random; it shows instead that a marginal-frequency test on its own is not sufficient to assess randomness.

Lag-1 autocorrelation is a diagnostic signal of first-order dependence. The five per-asset medians are all positive: ETH $+0.745$, BTC $+0.636$, DOGE $+0.477$, BNB $+0.366$, SOL $+0.095$ (Table 4.2). Their magnitude broadly co-varies with bps but is not strictly monotone; on a per-month basis (Fig. 4.1, top-right) the distribution stays well above zero on BTC, ETH, DOGE, and BNB, and is close to zero on SOL. The role of this result is not to claim that activity determines autocorrelation, but to give an intuitive diagnostic of the non-independence of the raw tick stream: even when the marginal frequency is close to 0.5, adjacent bits are still positively correlated. The pattern is consistent with persistent same-side order flow — when a large taker order walks the book it can produce several same-direction trades in a row, and the long memory of trade-sign

autocorrelation (Lillo and Farmer, 2004) adds further positive correlation. Lag-1 autocorrelation here therefore serves mainly as a diagnostic indicator of short-range dependence in the raw tick stream.

The longest run is a structural cryptographic risk signal. The $L_{\max}$ column of Table 4.2 reports the largest run length observed across the 15 monthly cells per asset: BTC 12,039, ETH 15,899, DOGE 5,071, BNB 2,069, SOL 792. The i.i.d. random expectation is $E[L_{\max}] \approx \log_2 n$, which for the monthly bit-stream lengths in this thesis ($n \sim 10^6$ to 4×10^7) is about 20 to 25. Every observed $L_{\max}$ is two to three orders of magnitude above this baseline. As an extreme statistic over 15 months, $L_{\max}$ fluctuates considerably and its ordering is not strictly the same as the bps ordering (ETH above BTC, DOGE above BNB); but "far above the i.i.d. expectation" holds without exception for all five assets. The longest run is therefore not used as a standalone acceptance criterion, but as an intuitive explanation of how the raw tick stream fails as a cryptographic random source: if the sampling window for a seed or password falls inside such a long run, the output degenerates into a long block of all 0s or all 1s.

The bit emission rate sets an upper bound on downstream throughput. Per-cell bit rates range from 0.68 bps (DOGE, 2026-03) to 15.52 bps (BTC, 2026-02); the per-asset medians are BTC 10.68, ETH 9.56, BNB 2.49, DOGE 2.02, SOL 1.74 bps (Table 4.2). The top two match the raw trades/s ranking in Table 4.1 (BTC then ETH), but the bottom three are reversed: although SOL has a higher raw trades/s than BNB and DOGE, its zero-delta ratio is the largest of the five (median ≈ 0.58 across the 15 months), so after dropping the zero-difference events SOL is left with fewer bit events than BNB (zero-delta ratio ≈ 0.44) and ends up last in the bps ranking. The baseline throughput therefore depends not only on trades/s but also on the per-asset zero-delta ratio, and it forms the upper bound for any post-aggregation throughput estimate.

4.2.3 Implications

Experiment 1 gives a negative but quantitative conclusion: the sign-encoded raw tick sequence cannot be used as an RNG. Although the marginal frequency is nearly balanced, Runs and the conditional-predictability battery reject all 75 (asset, month) cells; the lag-1 autocorrelation and the longest run further show that the failure comes from adjacent dependence and long structure, not merely from a shift in the 0/1 ratio. Monobit's misses on the lower-activity assets (22 of the 75 cells fail to reject) show that later experiments cannot rely on a marginal-frequency test alone. This conclusion provides three direct anchors for Experiment 2.

1. **Aggregation is necessary.** The $\ell = 1$ raw tick stream is rejected on every cell by Runs, ApEn, and D, so Experiment 2 must reduce the conditional dependence through aggregation rather than search for an acceptable window in the raw sign stream.

2. **First-order residual structure as a diagnostic comparator.** The lag-1 autocorrelation is positive on all five assets, which shows a stable first-order dependence in the raw tick sign stream; its magnitude broadly co-varies with bps, which suggests the diagnostic needs to cover a range of activity levels. Experiment 2 therefore uses it as a comparator between the transaction-time and one-second-bar aggregation axes. The later experiments do not assume this structure will be fully removed; instead they test how far each aggregation axis can reduce this short-range dependence.

3. **Upper bound on throughput.** The baseline bits/s scale across the 75 monthly cells (1–16 bps) is the ceiling for the throughput metric in § 3.7 and for the entropy-rate discussion

in Experiment 3; any later aggregation trades throughput below this bound for stronger randomness diagnostics.

4.3 Experiment 2: Aggregation

Experiment 2 answers the positive side of RQ1. Experiment 1 has shown that the raw tick stream is not usable, so aggregation is necessary; this experiment tests how deep the aggregation must be on each of the two time axes to make the bit stream pass standard randomness tests at $\alpha = 0.01$.

This section proceeds in four steps. Each step exposes a problem that the next step then addresses.

1. **Transaction-time single offset** (§ 4.3.2): the starting offset is fixed at $o = 0$. This is a quick pilot step: it gives the first estimate of the size of ℓ^*, but it does not decide final acceptance.
2. **Transaction-time all-offset strict criterion** (§ 4.3.3): the analysis moves to the full all-offset construction and uses the strict 80% pass-rate criterion to test robustness across offsets.
3. **Transaction-time all-offset relaxed criterion** (§ 4.3.4): the rule is loosened so that a small number of offsets is enough to pass. This checks whether the strict rule is rejecting too many otherwise usable cells.
4. **Physical-time 1-second bars** (§ 4.3.5): the analysis switches to the physical-time axis and checks whether the short-range dependence behaves differently when the data is sampled once per second.

§ 4.3.6 then collects the implications from the four steps and gives the configuration that downstream experiments will use.

4.3.1 Setup

The shared setup below covers §§ 4.3.2 to 4.3.5. Each subsection only states the differences in the ℓ grid.

- **Asset universe.** The same five USDT spot pairs as Experiment 1 (`BTCUSDT`, `ETHUSDT`, `BNBUSDT`, `SOLUSDT`, `DOGEUSDT`; see § 4.1).
- **Window.** The same 15-month sample (2025-01 to 2026-03). The diagnostic granularity is monthly: one bit stream per (asset, month) cell, $5 \times 15 = 75$ cells in total.
- **Encoding.** Algorithm 1: on the transaction-time axis ℓ has the unit "number of trades"; on the physical-time axis the pipeline in § 3.5 first turns the price series into one closing price per second, and ℓ becomes "number of seconds". The drop-zero step is the same on both axes.
- **All-offset construction.** § 3.2: for each ℓ, ℓ offset bit streams are built and tested independently. § 4.3.2 first uses the simplified single-offset version ($o = 0$ only, one stream); from § 4.3.3 on, the full all-offset version is used.
- **Acceptance criteria.** The main acceptance rule is D (adaptive k) + Monobit: the single-offset version asks for both p-values to be $\geq \alpha$; the all-offset strict criterion asks for both pass rates to be ≥ 0.80; the relaxed criterion is the heuristic version in § 3.4. Runs, $D(k = 2)$, ApEn, and the Shannon bias are still recorded as auxiliary diagnostics. Runs is also added explicitly in § 4.3.5 as the "+Runs" criterion.

– ℓ **grid.** § 4.3.2 and § 4.3.3 use ℓ from 50 to 2000 with step 25; § 4.3.4 uses 10 to 2000 with step 2; § 4.3.5 uses 10 to 600 with step 1.

4.3.2 Transaction-Time: Single Offset

This step uses the simplified single-offset version with $o = 0$: for each ℓ, only one bit stream is built, and the basic relation between aggregation depth and the pass result is inspected. The selected ℓ^* is the smallest ℓ at which D (adaptive k) and Monobit both give $p \geq \alpha$ ($\alpha = 0.01$). The ℓ grid is 50 to 2000 with step 25, applied to all $5 \times 15 = 75$ (asset, month) cells.

Table 4.3: Single-offset criterion – per-asset summary.

Asset	n_pass	selected ℓ^* median (over n_pass)	selected ℓ^* range
BTC	13/15	1300	[900, 1975]
ETH	15/15	725	[525, 1575]
BNB	15/15	275	[175, 850]
SOL	15/15	225	[100, 750]
DOGE	15/15	225	[75, 825]

n_pass is the number of months (out of fifteen) on which the single-offset criterion returns acceptable; the median and the range of selected ℓ^* are computed only over those acceptable months. Rows are sorted by median in descending order. The full per-(asset, month) table is in Table A.1.

Selected ℓ^* shows a coarse activity-based layering. The per-asset medians (BTC $\approx$ 1300, ETH $\approx$ 725, BNB $\approx$ 275, SOL $\approx$ 225, DOGE $\approx$ 225) show that BTC and ETH need a clearly deeper aggregation in the $[50, 2000]$ range, while the three lower-activity assets reach the threshold around 200–300. This is in the same direction as the "higher trades/s, larger required ℓ" relation reported by Onofri et al. (2025) on 8 US stocks plus 1 ETF; but the result should be read only as a coarse layering, not as a strict monotone ranking between all five assets.

Single offset is used as a pilot diagnostic; final acceptance is determined by the all-offset criterion. In 2025-01 and 2025-08, BTC does not let D and Monobit pass together for any ℓ in $[50, 2000]$ (see Table A.1). This shows that fixing $o = 0$ can give an approximate size for ℓ^*, but it cannot show whether other starting offsets behave in the same way. These failures also cannot be explained only by single-month trades/s.

The next question is whether the result is stable across offsets. For a fixed ℓ, the all-offset construction has ℓ possible starting offsets. Passing at $o = 0$ does not mean that the other $\ell - 1$ offset streams also pass. If the result changes strongly with the starting offset, the cell should not be accepted only because one offset works. § 4.3.3 therefore upgrades the pilot diagnostic to the full rule: at least 80% of offsets must pass together.

4.3.3 Transaction-Time: All-Offset Strict Criterion

This step replaces the "single offset passes" rule with the strict criterion of the all-offset framework (Algorithm 2, § 3.4): for each ℓ, ℓ offset bit streams are built, and at least 80% of them must give $p \geq \alpha$ on both D (adaptive k) and Monobit at the same time ($\alpha = 0.01$). The selected ℓ^* is the smallest ℓ on which this holds. The ℓ grid stays at 50 to 2000 with step 25.

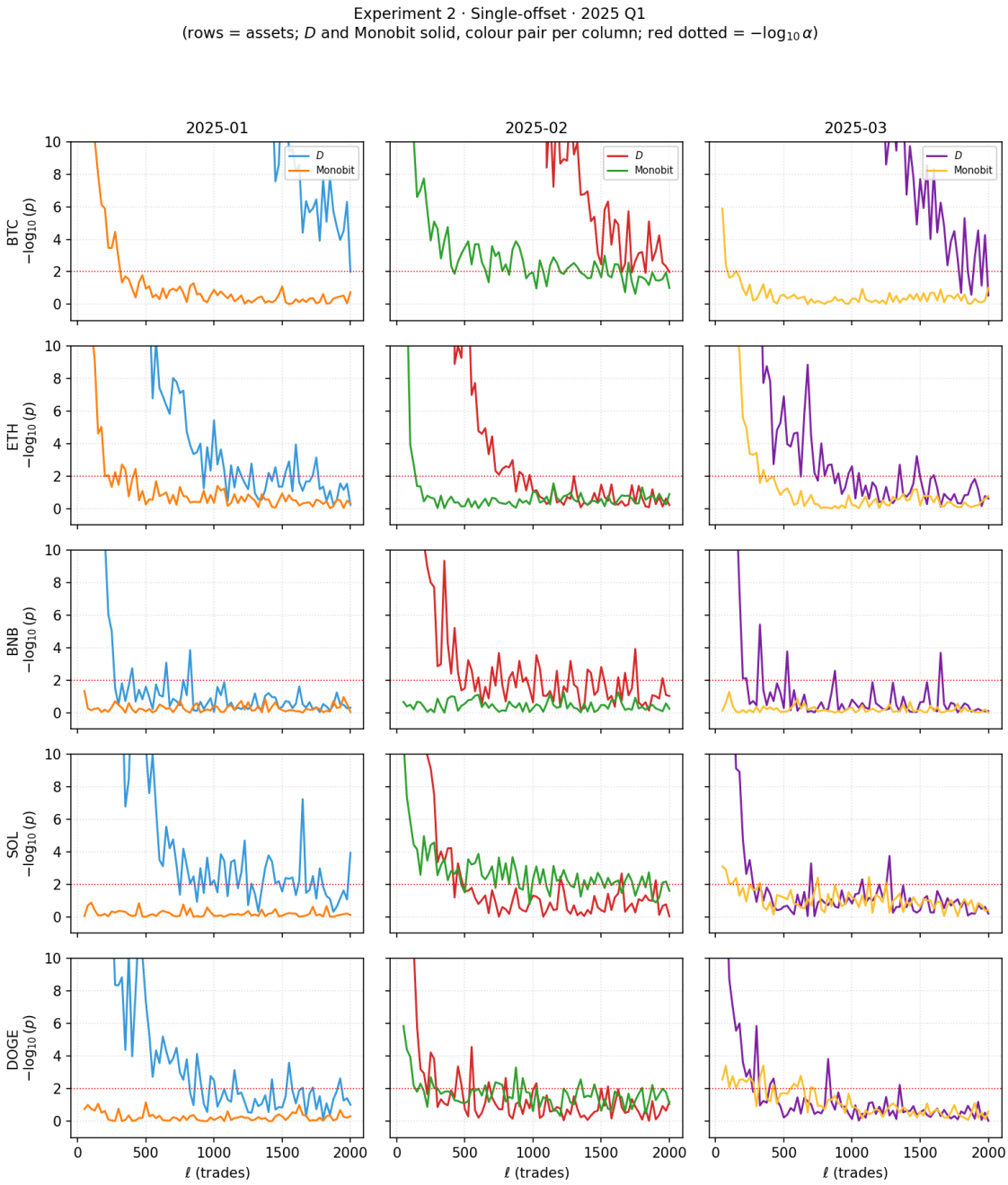

Figure 4.2: Single-offset $-\log_{10}(p)$ versus ℓ on 2025 Q1.

Each panel arranges the curves by asset and by month, showing how the $-\log_{10}(p)$ of D and Monobit drops as ℓ grows. The horizontal red line marks the $\alpha = 0.01$ threshold. The full 15-month table of selected ℓ^* is in Table A.1.

The strict criterion pushes ℓ^* higher; the coarse ranking remains the same as in single offset. Compared with § 4.3.2, the strict criterion raises the per-asset median of selected ℓ^* on every asset: BTC $1300 \to 1500$, ETH $725 \to 975$, BNB $275 \to 350$, DOGE $225 \to 375$, SOL $225 \to 350$ (the two medians are computed over different months because the acceptable subsets differ, so the comparison is a magnitude trend, not a month-by-month difference). This is the cost of asking 80% of offsets to pass together rather than only one: the joint confirmation from many offsets is

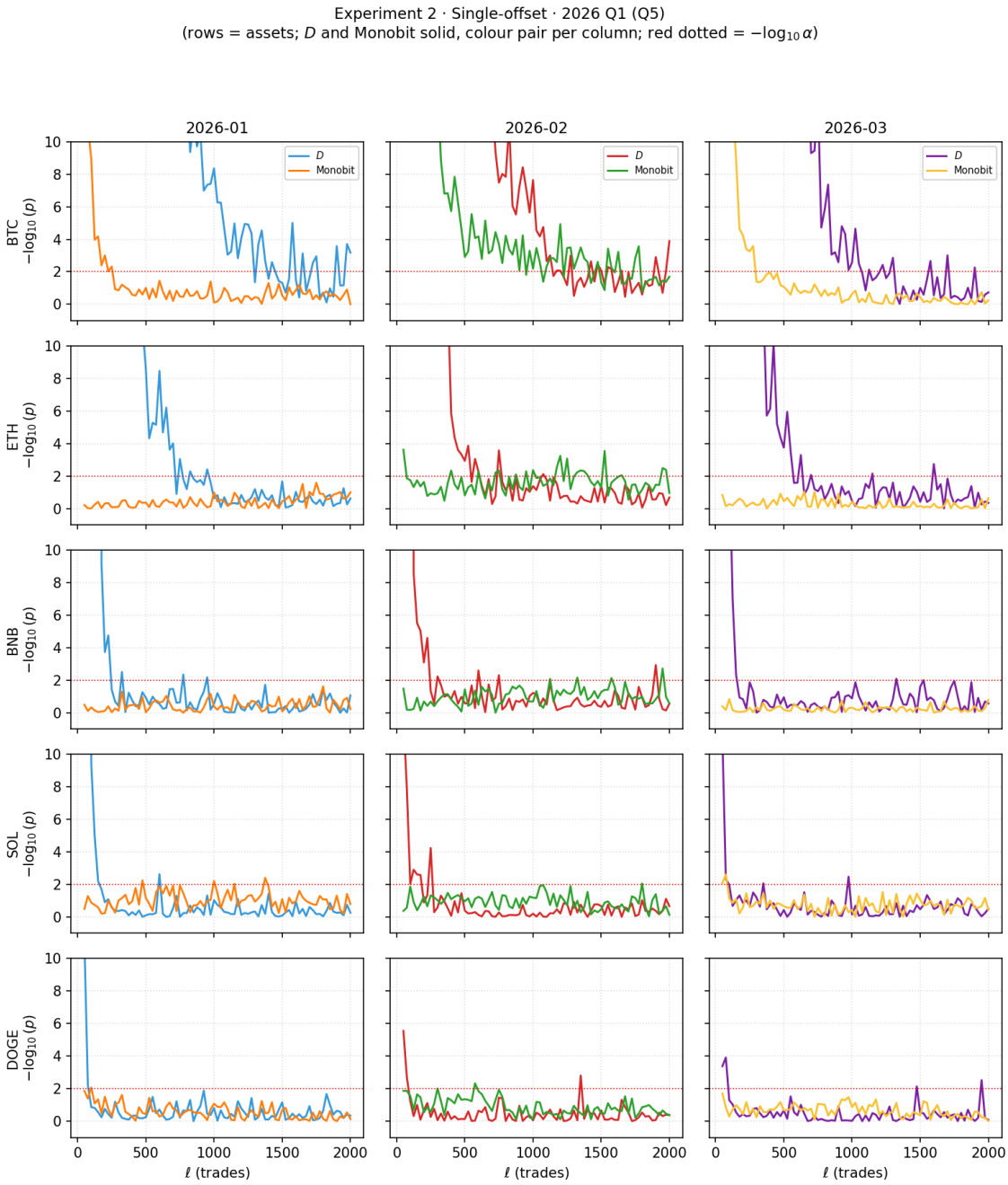

Figure 4.3: Single-offset $-\log_{10}(p)$ versus ℓ on 2026 Q1 (Q5).

Same format as Fig. 4.2.

statistically a higher bar, so a deeper aggregation is needed to move out of the rejection region. The relation between assets is still that BTC and ETH are clearly above BNB, SOL and DOGE; the lower three should not be read as a strict monotone ranking.

BTC plays the high-activity outlier role in this dataset: 8 of 15 months have no solution in $\ell \leq 2000$. Under single offset only 2 months (2025-01, 2025-08) fail; under the strict criterion this number rises to 8. The same asset and the same aggregation range now cost much more, mostly

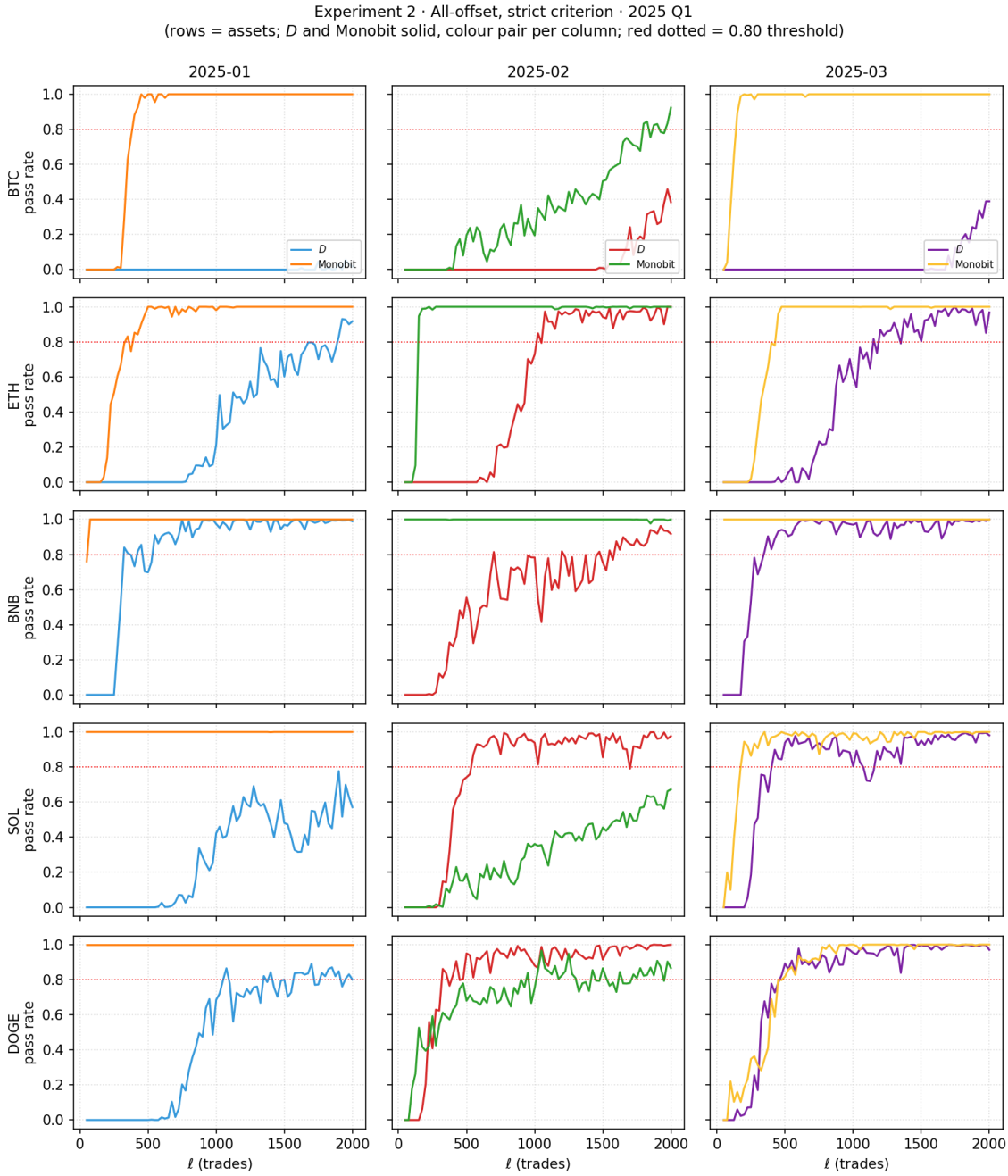

Figure 4.4: All-offset strict criterion – pass rate versus ℓ on 2025 Q1.

Each panel shows the offset pass rate of D and Monobit for one (asset, month) cell. The horizontal red line is the 0.80 strict criterion threshold. The full 15-month table of selected ℓ^* is in Table A.2.

because BTC has to carry the 80% joint confirmation. This failure pattern cannot be explained by single-month raw trades/s alone: ETH, which also has high activity, still passes 13 of 15 months, while BTC fails on both lower and higher activity months. ETH and SOL each have 2 failing months, so 63 of 75 cells pass and the overall acceptance rate of the strict criterion is $63/75 = 84\%$.

The streams that pass the strict criterion still fail Runs in a systematic way. The strict criterion uses only D (adaptive k) and Monobit as its main test, with Runs and $D(k = 2)$ kept as

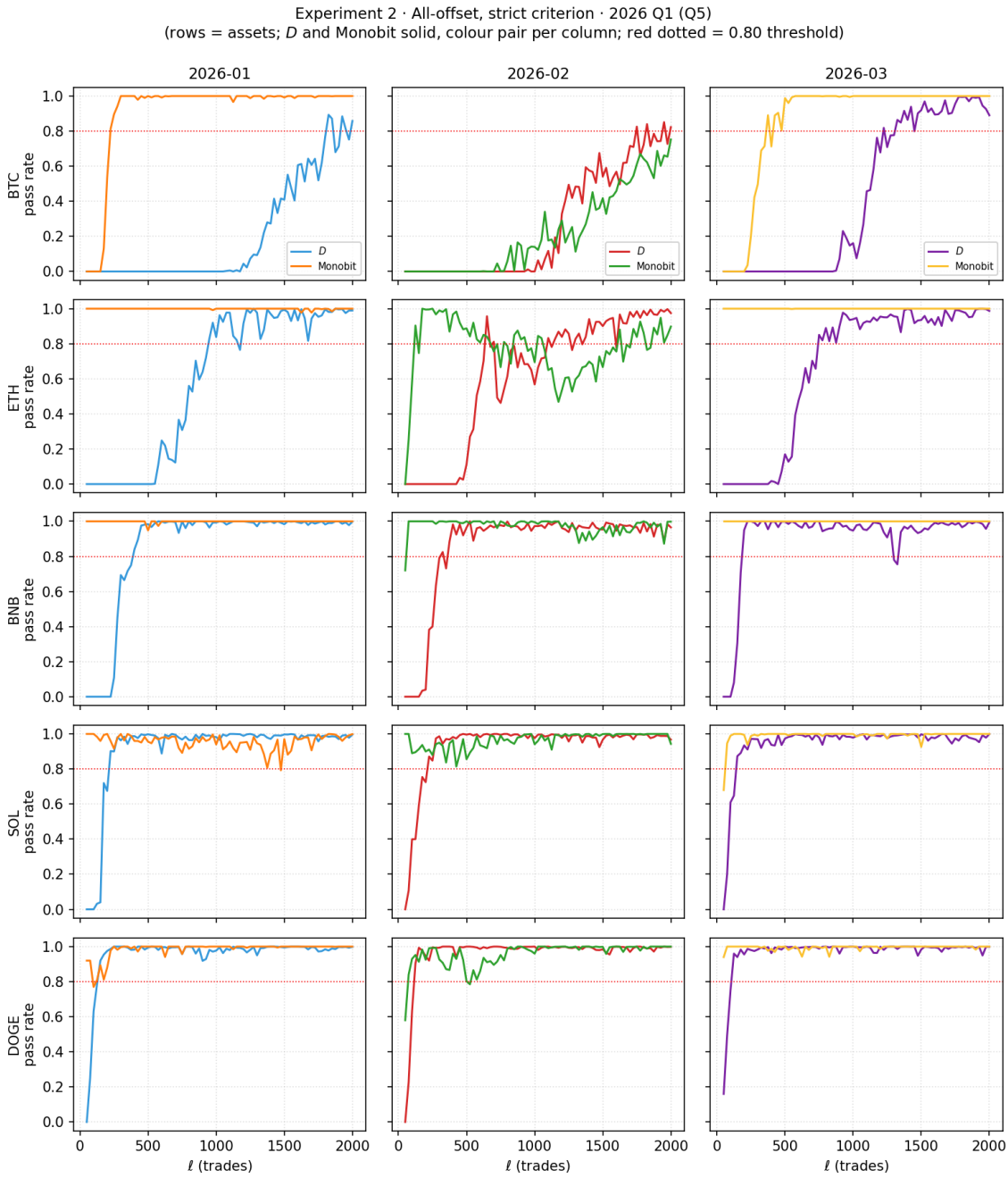

Figure 4.5: All-offset strict criterion – pass rate versus ℓ on 2026 Q1 (Q5).

Same format as Fig. 4.4.

diagnostics. Among the 63 cells accepted, 38 cells have a Runs pass rate below 0.05 (see the note of Table A.2): at the selected ℓ^* of those cells almost every offset stream is rejected by Runs, and $D(k = 2)$ shows the same pattern. Passing the strict criterion therefore certifies D (adaptive k) and Monobit, while first-order structure can still remain. This residual is a stable feature on the transaction-time axis, and is one reason for comparing the physical-time axis in § 4.3.5.

The strict criterion leaves two practical problems. First, BTC has no acceptable $\ell \leq 2000$ on

Table 4.4: All-offset strict criterion – per-asset summary.

Asset	n_pass	selected ℓ^* median (over n_pass)	selected ℓ^* range
BTC	$7/15$	1500	$[1225, 1825]$
ETH	$13/15$	975	$[650, 1900]$
DOGE	$15/15$	375	$[75, 1075]$
SOL	$13/15$	350	$[150, 650]$
BNB	$15/15$	350	$[200, 1025]$

n_pass is the number of months (out of fifteen) on which the strict criterion returns acceptable; the median and the range of selected ℓ^* are computed only over those acceptable months. Rows are sorted by median in descending order. The full per-(asset, month) table is in Table A.2.

8 months, which shows that the 80% requirement can be too strict for the highest-activity asset. Second, the month-to-month range of selected ℓ^* is wide, and the coarse grid further limits the precision of the estimate. § 4.3.4 therefore tries a relaxed offset-coverage criterion and also refines the ℓ grid to step 2.

4.3.4 Transaction-Time: All-Offset Relaxed Criterion

This step replaces the strict criterion with the relaxed criterion defined in § 3.4. The main text uses $(F, f) = (3, 0.03)$, and the ℓ grid is refined to 10 to 2000 with step 2.

Table 4.5: All-offset relaxed criterion – per-asset summary.

Asset	n_pass	selected ℓ^* median	selected ℓ^* range
BTC	$15/15$	1150	$[870, 1750]$
ETH	$15/15$	556	$[400, 1194]$
BNB	$15/15$	212	$[128, 464]$
SOL	$15/15$	180	$[66, 596]$
DOGE	$15/15$	156	$[48, 594]$

n_pass is the number of months on which the relaxed criterion returns acceptable; the median and the range are computed over all 15 months. Rows are sorted by median in descending order. The full per-(asset, month) table is in Table A.4.

The relaxed criterion raises coverage from $63/75$ to $75/75$. The strict criterion fails on 8 months of BTC, 2 months of ETH and 2 months of SOL, which is 12 cells in total (Table 4.4). The relaxed criterion finds an acceptable ℓ^* on all 75 cells. The 8 BTC months that had no $\ell \leq 2000$ acceptable under the strict criterion now find $\ell^* \in [998, 1750]$ under the relaxed criterion, all still inside the same $[50, 2000]$ grid; it is the strict 80% bar that is too tight on those cells. This observation directly responds to the practical coverage problem raised at the end of § 4.3.3.

Selected ℓ^* drops overall, but the coarse asset layering does not change. A per-asset comparison (strict over its acceptable months, relaxed over all 15 months) shows that the relaxed criterion pulls every per-asset median down: BTC $1500 \to 1150$, ETH $975 \to 556$, BNB $350 \to 212$, DOGE $375 \to 156$, SOL $350 \to 180$, which is a reduction of about 23% (BTC) to 58% (DOGE). The two

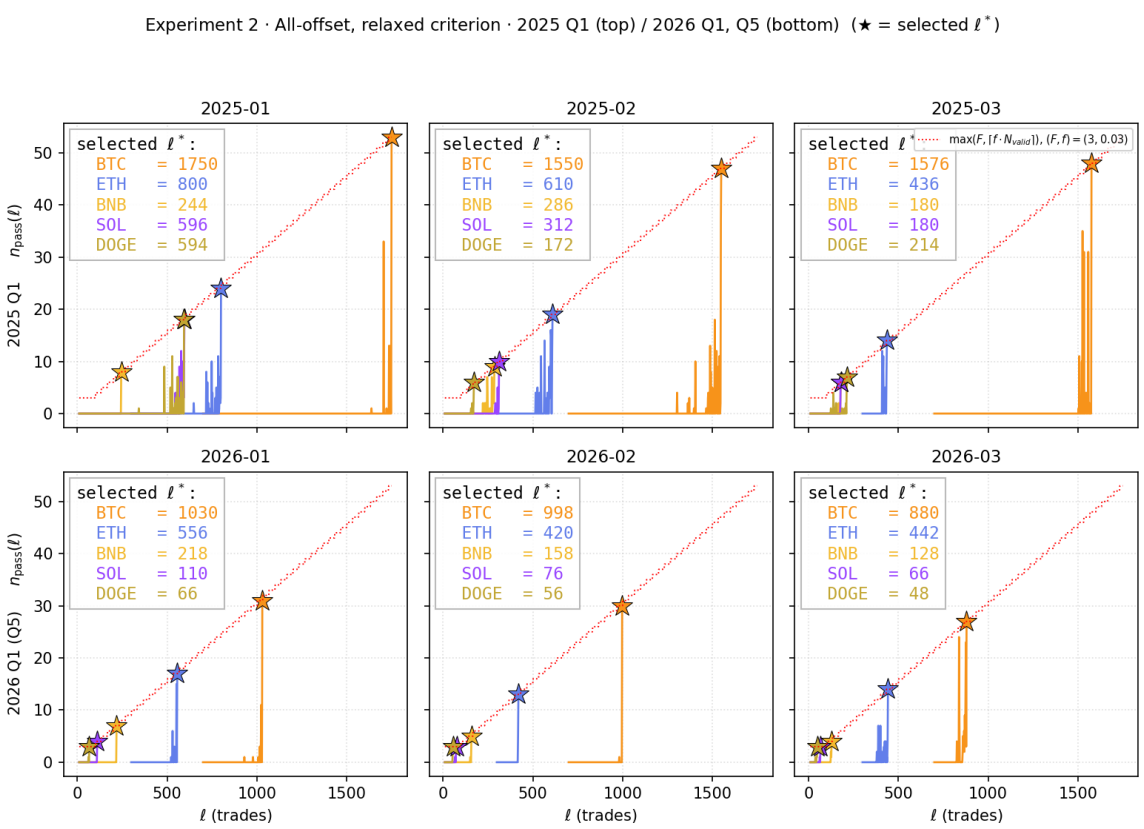

Figure 4.6: All-offset relaxed criterion – $n_{\text{pass}}(\ell)$ and selected ℓ^*.

2025 Q1 and 2026 Q1 (Q5) are shown together. Each panel plots $n_{\text{pass}}(\ell)$ for the five assets, where n_{pass} is the number of offsets that pass both D and Monobit. The red line is the relaxed criterion threshold; the marked points are the selected ℓ^*. The full 15-month results are in Table A.4; a Bonferroni comparison is in Table A.5.

medians use different month bases (strict has 12 failing cells, relaxed has 75 successful cells), so the reduction is a magnitude comparison rather than a month-by-month difference. The relaxed criterion is a looser bar, so every asset passes at a smaller ℓ.

The accepted-cell descriptive throughput of the relaxed criterion is higher than that of the strict criterion. On the accepted cells, the median bit rate of the selected output offset stream rises from about 0.012 bps under the strict criterion to about 0.021 bps under the relaxed criterion, which is a factor of about $1.8\times$. This difference mainly comes from the lower ℓ^* values that the relaxed criterion selects; but it is only a descriptive comparison between the two criteria on this experiment's cell set, not the deployment throughput of any multi-asset system. The advantage of the relaxed criterion should therefore be read as more complete coverage and a higher per-cell bit rate; the cost is that the robustness argument moves from "80% of offsets pass together" to "at least 3 offsets pass together", and the latter is explicitly marked as a heuristic rather than a formal correction.

Bonferroni / Šidák comparison gives smaller ℓ^*. As a directional comparison, a Bonferroni / Šidák version on 2025 Q1 gives systematically smaller selected ℓ^* than the heuristic relaxed criterion. This is consistent with the discussion in § 3.4, so it is not used as the main criterion; the full results are in Table A.5.

This subsection closes the chain on the transaction-time axis: single offset → strict criterion →

relaxed criterion. The last two trade off offset stability, coverage and bit rate. The relaxed criterion gives full $75/75$ coverage, but it is still a heuristic and does not give a stronger statistical guarantee. One important question remains: $D(k = 2)$ and Runs are almost always rejected near the selected ℓ^*, so the short-range dependence may be partly caused by the sampling axis. § 4.3.5 therefore switches to the physical-time axis (1-second bar).

4.3.5 Physical-Time: 1-Second Bars

This step redoes the all-offset acceptance loop on the physical-time axis. Different from transaction-time, the price series is first turned into a per-second closing price, and then ℓ takes the unit "seconds" (see § 3.5). The ℓ grid is 10 to 600 with step 1; the criterion still uses the 80% pass-rate framework. The base criterion already gives $70/75$ coverage on 1-second bars, so no relaxed criterion is needed here.

Table 4.6: 1-second bar criterion – per-asset summary.

Asset	base n_pass	base ℓ^* median (sec)	+Runs n_pass	+Runs ℓ^* median (sec)
BNB	$14/15$	84	$14/15$	101
BTC	$15/15$	62	$15/15$	92
DOGE	$14/15$	33	$13/15$	53
ETH	$12/15$	65	$8/15$	56
SOL	$15/15$	54	$12/15$	59

Each row aggregates 15 (asset, month) cells under two criteria: base (D + Monobit) and +Runs. For even sample sizes the ℓ^* median is the average of the two middle values rounded to the nearest integer (BNB base $83.5 \to 84$, ETH base $64.5 \to 65$, ETH +Runs $55.5 \to 56$, SOL +Runs $58.5 \to 59$). The full per-(asset, month) tables, including the +Runs+ApEn criterion, are in Tables A.6 to A.8.

The 1-second bar base criterion covers $70/75$ cells on the physical-time axis. The base criterion gives 70 acceptable cells out of 75, which shows that the axis change still finds a second-level ℓ^* on most cells. The failures concentrate on a few months of ETH, BNB and DOGE; the seconds-with-trades coverage can explain a part of these failures, but it is not the only factor.

Adding Runs to the criterion moves the coverage from $70/75$ to $62/75$, which is the key independence result on the 1-second bar axis. To compare the two axes under the same criterion, Runs is also added to the transaction-time strict criterion and to the 1-second bar criterion. Transaction-time strict+Runs covers only $48/75$ cells (Table A.3), while 1-second bar +Runs covers $62/75$; adding ApEn after that keeps the same $62/75$. This comparison shows that the first-order residual which is hard to handle on the transaction-time axis is clearly reduced on the physical-time axis.

This independence improvement is connected to the aggregation axis. The same aggTrades data still keeps a strong first-order residual on the transaction-time axis, but on the physical-time axis most cells get a clear reduction. This result does not support the reading that "the first-order residual comes entirely from the source data and cannot be reduced"; a more reasonable reading is that sampling by equal trade counts concentrates same-direction order flow into adjacent bits, while a 1-second bar spreads out this local concentration. This is consistent with the Experiment 1 diagnostic that lag-1 autocorrelation is positive on all five assets, with a magnitude broadly co-varying with activity.

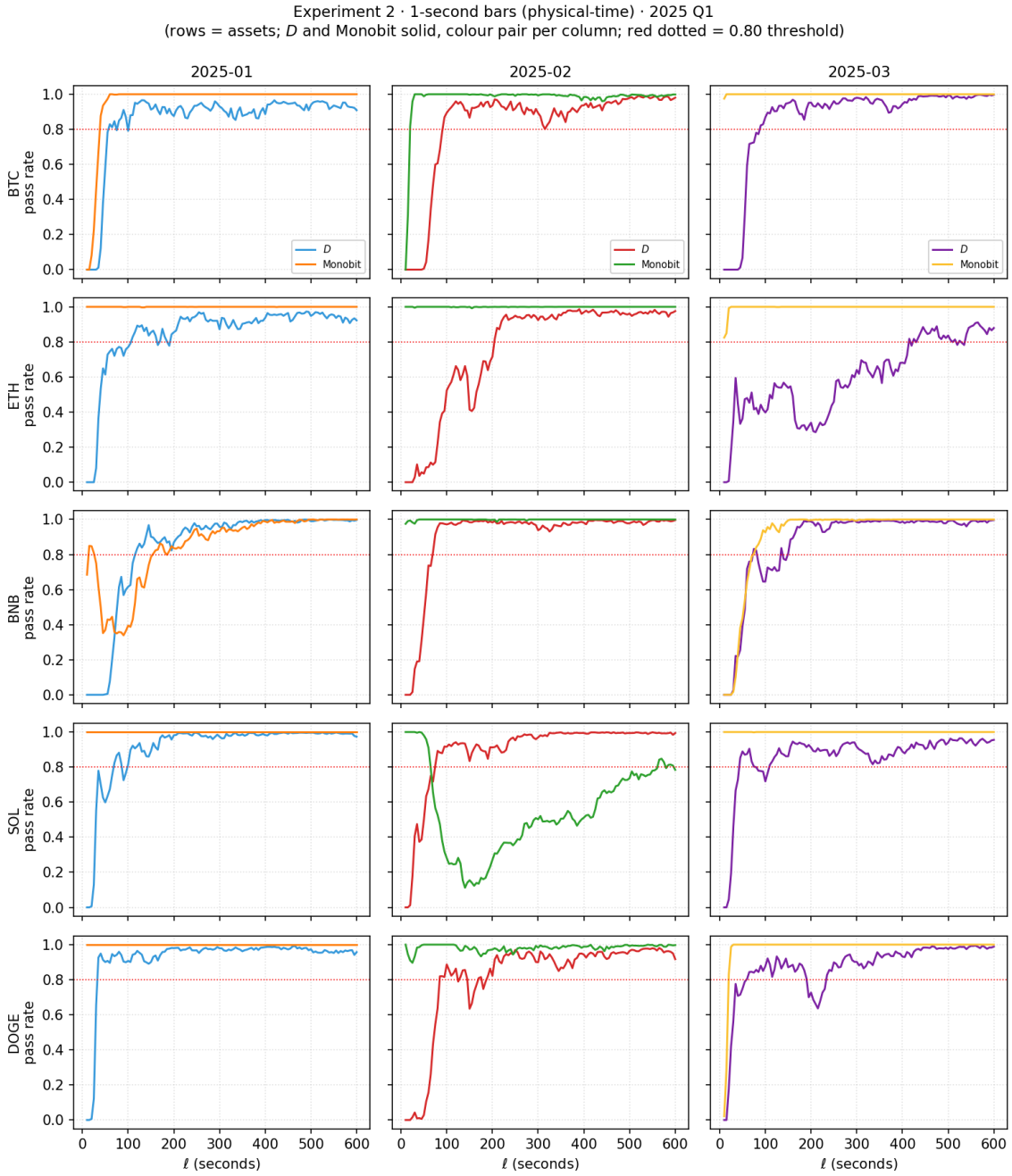

Figure 4.7: 1-second bar criterion – pass rate versus ℓ on 2025 Q1.

The format is the same as Fig. 4.4, but the unit of ℓ changes from "number of trades" to "seconds". The horizontal red line is the 0.80 pass rate threshold. The full 15-month tables of selected ℓ^* across the three criteria are in Tables A.6 to A.8.

The physical-time axis is easier to interpret as wall-clock latency. Transaction-time throughput varies with intra-month trades/s. On the physical-time axis, ℓ^* is measured in seconds, so it directly describes how long one waits before producing the next candidate bit. The drop-zero step still filters out zero-difference seconds, so the actual bit rate is a little below $1/\ell^*$, but the magnitude is similar. The descriptive rate corresponding to the ℓ^* medians in Table 4.6 falls roughly in the 0.01–0.03 bps range. Adding Runs reduces coverage, and on some assets and months it also raises

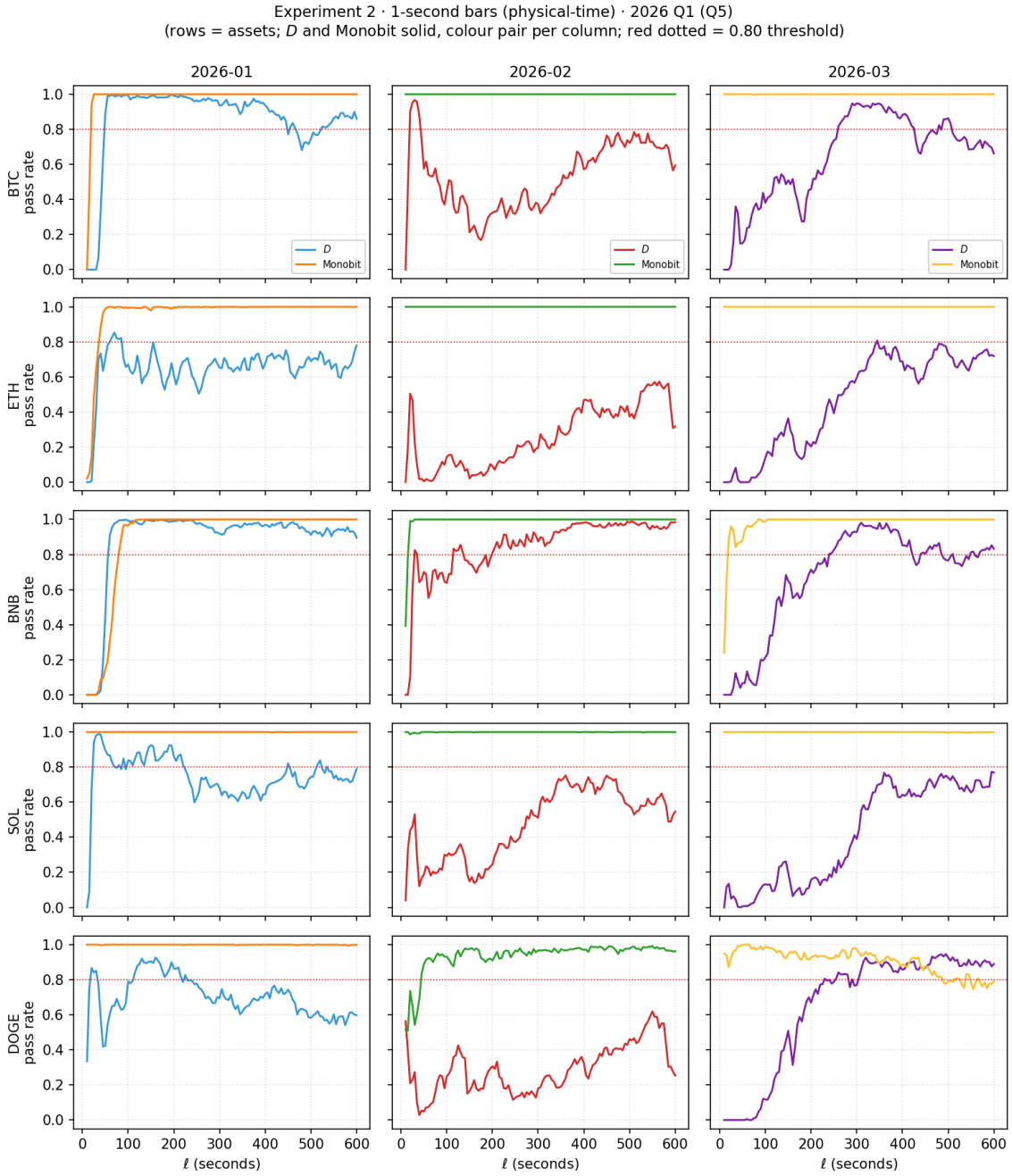

Figure 4.8: 1-second bar criterion – pass rate versus ℓ on 2026 Q1 (Q5).

Same format as Fig. 4.7.

the waiting time.

4.3.6 Implications

The main role of Experiment 2 is to choose the aggregation axis for the later pipeline. Both axes can produce accepted cells, but they have different practical meanings. Transaction-time is useful as a statistical comparison because it aggregates by number of trades. Physical-time is easier to use later because ℓ is measured in seconds. This subsection summarises the four conclusions.

1. **Physical-time should be used as the main aggregation axis.** The bit rate on transaction-time varies with trades/s, so it is more suitable as a statistical construction and as a supplementary comparison. In contrast, ℓ on physical-time is in seconds, and the selected ℓ^* can be read directly as a wall-clock sampling schedule and an approximate waiting time per candidate bit.

2. **Physical-time is more suitable as the input configuration for the extended tests.** Under the +Runs criterion, the transaction-time strict criterion drops from $63/75$ to $48/75$ accepted cells (a within-axis loss of 24%), while the 1-second bar base criterion drops from $70/75$ to $62/75$ accepted cells (a within-axis loss of 11%); adding ApEn after that keeps the same $62/75$. The within-axis coverage loss is clearly larger on transaction-time, which shows that the Runs / $D(k = 2)$ residual on transaction-time is at least partly amplified by the sampling axis. The later entropy-rate analysis, the additional NIST sub-tests, and the TestU01 sanity check therefore use the bit streams selected on the physical-time axis.

3. **Transaction-time relaxed criterion is kept as a supplementary configuration.** It shows that the trade-count axis can also reach full coverage when the offset-coverage rule is loosened. But this relaxed rule is heuristic, and its throughput varies with market activity, so it is not used as the main configuration. Experiment 3 also does not use it as a fallback: the audit target is the coverage boundary of the 1-second bar +Runs criterion itself. The 13 cells that fail under the 1-second bar +Runs criterion are therefore kept as evidence about that criterion's boundary and do not enter the Experiment 3 sample.

4. **Throughput is reported only as a magnitude anchor.** The per-cell bit rate of the selected cells in Experiment 2 is at the 10^{-2} bps order. This means that stronger randomness diagnostics come with a lower bit emission rate.

4.4 Experiment 3: Extended Test Battery

Experiment 2 selects the aggregation level ℓ^* on the 1-second bar axis using a small criterion: D (adaptive k), Monobit, and Runs must each reach an offset pass rate of at least 0.80. Under this +Runs criterion, 62 of the 75 (asset, month) cells are accepted. This criterion is enough to select an aggregation level, but it does not prove that the selected streams are random under a wider test battery.

Experiment 3 therefore audits the coverage boundary of the criterion in § 4.3.5: which structures does the small criterion detect, and which structures remain visible when the same streams are checked by a wider battery?

The 62 accepted cells are tested against the 29-sub-test universe defined in § 3.3. This battery adds NIST SP800-22 and TestU01 Alphabit items beyond the core tests already used in Experiment 2, so it gives a cross-battery check. The main sample of Experiment 3 is fixed to the +Runs criterion. The earlier base criterion (D + Monobit) is kept only as a sensitivity comparison, because § 4.3.5

has already shown that excluding Runs allows clear first-order structure to remain in the candidate streams.

4.4.1 Setup

- **Sample.** The sample contains the 62 (asset, month) cells accepted by the 1-second bar +Runs criterion in Experiment 2. The 13 cells that fail that criterion do not enter Experiment 3, and no fallback rescue is used. Experiment 3 also does not re-select ℓ^*; it audits each accepted cell at the ℓ^* already selected in § 4.3.5.
- **Battery.** The experiment uses the 29-sub-test universe from § 3.3. Of the five core tests reported around Experiment 2, only D (adaptive k), Monobit, and Runs form the +Runs criterion. $D(k = 2)$ and ApEn are diagnostic quantities and do not take part in the selection of ℓ^*. At short lengths, TestU01 Alphabit may skip long-block sub-tests; such (cell, sub-test) entries are marked as NOT_RUN and are not included in the admissible denominator.
- **Acceptance.** The acceptance rule follows Experiment 2. For each (cell, sub-test), the sub-test is counted as passed if at least 80% of valid offsets give $p \geq \alpha = 0.01$. Per-asset summaries report "passing months / admissible months".
- **Sanity check.** Before the main run, each sub-test is checked on IID null data from `/dev/urandom`. The sanity run uses $K = 1000$ trials over six length brackets: 5K, 10K, 25K, 50K, 100K, and 200K bits. These brackets match the bit-length range that cells can reach. The sanity check decides whether a (sub-test, bracket) is admissible in the main run. Passing entries enter the denominator; entries with too high an observed type-I rate are marked INVALID; entries that cannot run at that length are marked NOT_RUN.

4.4.2 Results

The sanity check shows that the extended battery is mostly usable. Out of $29 \times 6 = 174$ (sub-test, bracket) entries, 164 pass the sanity filter, with observed type-I rate $\leq 0.02 = 2\alpha$. The threshold is a loose sanity filter, not a re-calibration of the test level.[1] One entry is marked INVALID: LongestRun in the 5K bracket has type-I rate 0.122, which is consistent with the NIST length recommendation $N \geq 6272$ bits for LongestRun. Another 9 entries are marked NOT_RUN because the length is too short. All pass rates below use only sanity-admissible entries in the denominator.

Most sub-tests still pass under the extended battery. D (adaptive k), Monobit, Runs, and DFT pass in all admissible cells across all five assets. The first three are already part of the +Runs criterion. DFT is outside that criterion, so its 100% pass rate gives an additional spectral check. Most other sub-tests also pass on most months, as shown in Fig. 4.9. In this sense, the aggregation levels selected by Experiment 2 are broadly stable under a wider battery.

Three higher-order sub-tests expose the remaining weak point. Table 4.7 shows that Serial m has per-asset pass rates from 33% (SOL, $4/12$) to 64% (BNB, $9/14$), with $30/62$ passes in total. It is the weakest NIST SP800-22 extension in this experiment. Across the full 29-sub-test universe, MultinomialBitsOver L4 is even weaker, with per-asset pass rates from 25% to 57% and $28/62$ passes in total. ApEn also fails clearly, especially on ETH ($2/8$). These tests all target higher-order

[1] The threshold is set to 2α rather than α to allow binomial sampling variation under $K = 1000$ null trials. When $\alpha = 0.01$ and $K = 1000$, a correctly implemented sub-test has a binomial 99% confidence-interval upper end of about 0.018 (Clopper and Pearson, 1934). The value 0.02 is slightly above this.

Table 4.7: Experiment 3 – three sub-tests that fail across assets.

Sub-test	Battery	BTC	ETH	BNB	SOL	DOGE
Serial m	NIST SP800-22	$8/15$	$3/8$	$9/14$	$4/12$	$6/13$
MultinomialBitsOver L4	TestU01 Alphabit	$8/15$	$3/8$	$8/14$	$3/12$	$6/13$
Approximate Entropy	Core diagnostic	$11/15$	$2/8$	$8/14$	$9/12$	$9/13$

Each entry is "passing months / admissible months". The admissible month count differs by asset because the +Runs criterion accepts different numbers of monthly cells: BTC 15, ETH 8, BNB 14, SOL 12, and DOGE 13. Rows are grouped by source battery, not sorted by failure rate.

or multi-symbol structure. Their failures show that the small criterion from Experiment 2 removes much of the short-range dependence, but still leaves structure that the criterion does not see.

The two independent test libraries agree on the failure pattern. Serial m comes from NIST SP800-22, while MultinomialBitsOver L4 comes from TestU01 Alphabit. They show almost the same per-asset failure pattern. This is the most important observation of Experiment 3: two independent test libraries confirm the same residual signal.

The base-criterion comparison supports the +Runs choice in Experiment 2. As a sensitivity check, the 70 cells selected earlier by the base criterion (D + Monobit) are also tested with the same extended battery at their selected ℓ^*. Compared with the +Runs main sample, the base-selected streams are much worse on $D(k = 2)$ and Runs. Their cross-asset average pass rates fall from almost 100% under +Runs to about 29%, with BTC, ETH, SOL, and DOGE near 20% and only BNB higher. This confirms that adding Runs to the main criterion is not an arbitrary tightening. It prevents clear first-order structure from entering the candidate streams.

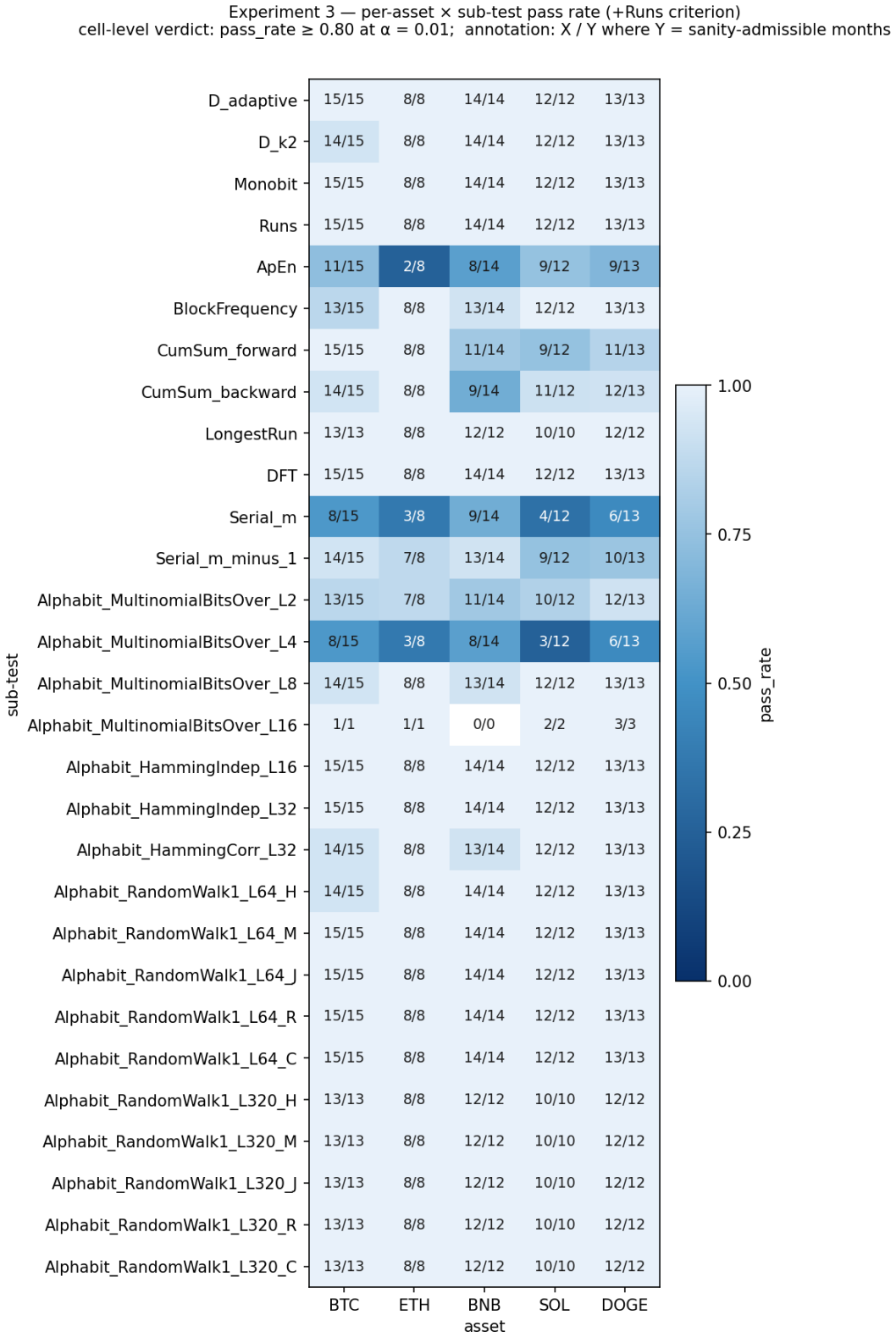

Figure 4.9: Experiment 3 – per-asset sub-test pass rates.

Each cell shows the share of admissible months on which one asset passes one sub-test. The main sample is the 62-cell 1-second bar +Runs sample selected in Experiment 2.

4.4.3 Implications

Experiment 3 gives a more detailed conclusion than Experiment 2. Single-asset aggregation on the 1-second bar axis strongly reduces short-range dependence: D (adaptive k), Monobit, Runs, and DFT all pass in the extended battery, and $D(k = 2)$ fails in only one of the 62 cells (BTC, $14/15$). However, a higher-order residual remains. Serial, ApEn, and the multi-symbol Alphabit tests can still detect it, and the pattern is confirmed by both NIST SP800-22 and TestU01.

This result motivates Experiment 4. If a single aggregated asset stream still contains higher-order residual structure, then combining several asset streams may reduce the residual further. The intuition and limits of XOR combination are given in § 3.6; Experiment 4 only tests whether it works on the held-out months, and how much throughput it costs.

4.5 Experiment 4: Multi-Asset XOR Combination

Experiment 3 shows that single-asset aggregation strongly reduces short-range dependence, but Serial, ApEn, and multi-symbol Alphabit tests still detect higher-order residuals. Experiment 4 therefore moves from the question "is one aggregated asset stream enough?" to a second question: can several asset streams be combined to reduce these residuals, and what throughput cost does this create?

This experiment answers the second layer of RQ3. It checks whether multi-asset combination gives, in an out-of-sample validation window, three things at the same time: stronger statistical robustness, acceptable throughput, and enough estimated min-entropy to support the downstream password generator. The main result is that $n \in \{2, 3, 5\}$ gives deployable streams, while $n = 4$ fails systematically in validation. This failure is not a side note; it is a useful methodological result about calibration under non-stationary market signs.

§ 3.6 already states the intuition and limits of XOR combination: under independent inputs XOR attenuates the marginal bias, but cryptocurrency asset sign streams are correlated, so this thesis treats it only as an assumption that must be checked. This section tests that assumption on the held-out months, and reports validation robustness, throughput, and min-entropy together.

4.5.1 Setup

- **Input.** The experiment uses the multi-asset combination and ℓ-level XOR aggregation defined in § 3.6. The operator is instantiated on the five 1-second sign-bit streams from BTC, ETH, BNB, SOL, and DOGE.
- **Calibration / validation split.** Calibration uses 2025-01 to 2025-09 (9 months). Validation uses 2025-10 to 2026-03 (6 months). Calibration selects the asset subset, ℓ_n^*, and the selected output offset. Validation keeps these choices fixed and does not tune parameters again.
- **Subset universe.** For each $n \in \{2, 3, 4, 5\}$, all $\binom{5}{n}$ asset subsets are enumerated. In total this gives $\binom{5}{2} + \binom{5}{3} + \binom{5}{4} + \binom{5}{5} = 10 + 10 + 5 + 1 = 26$ subsets.
- **Calibration search.** For each subset and each calibration month, the combined stream is built and the all-offset +Runs criterion is scanned over $\ell = 1, \ldots, 400$ with step 1. Here ℓ is the bit-domain XOR aggregation level from § 3.6, not the price-domain physical-time aggregation level used in § 4.3.5. The criterion still requires D (adaptive k), Monobit, and Runs to each reach pass rate at least 0.80.

- **Selection rule.** For each n, the subset with the highest estimated bits/month is selected. For that subset, ℓ_n^* is the 80th percentile of the nine monthly ℓ^* values. The 80th percentile gives more room than the median for month-to-month variation, but avoids letting one outlier month determine the whole setting. To fix the output stream, the selected output offset is chosen in the last calibration month: among valid offsets, it is the offset with the largest geometric mean of the three p-values from D (adaptive k), Monobit, and Runs. This selected output offset is fixed in validation. The validation battery pass/fail result is still based on all-offset pass rates, not on the selected output offset alone.
- **Validation battery.** Validation runs the 29-sub-test universe from § 3.3 on the four calibration-selected subsets, one for each n. Running all 26 subsets again in validation would leak test data into model selection. For each (month, sub-test), the verdict is pass if at least 80% of valid offsets pass at $\alpha = 0.01$; length-skipped or sanity-excluded items are not included in the admissible denominator.

4.5.2 Mutual Information Diagnostic

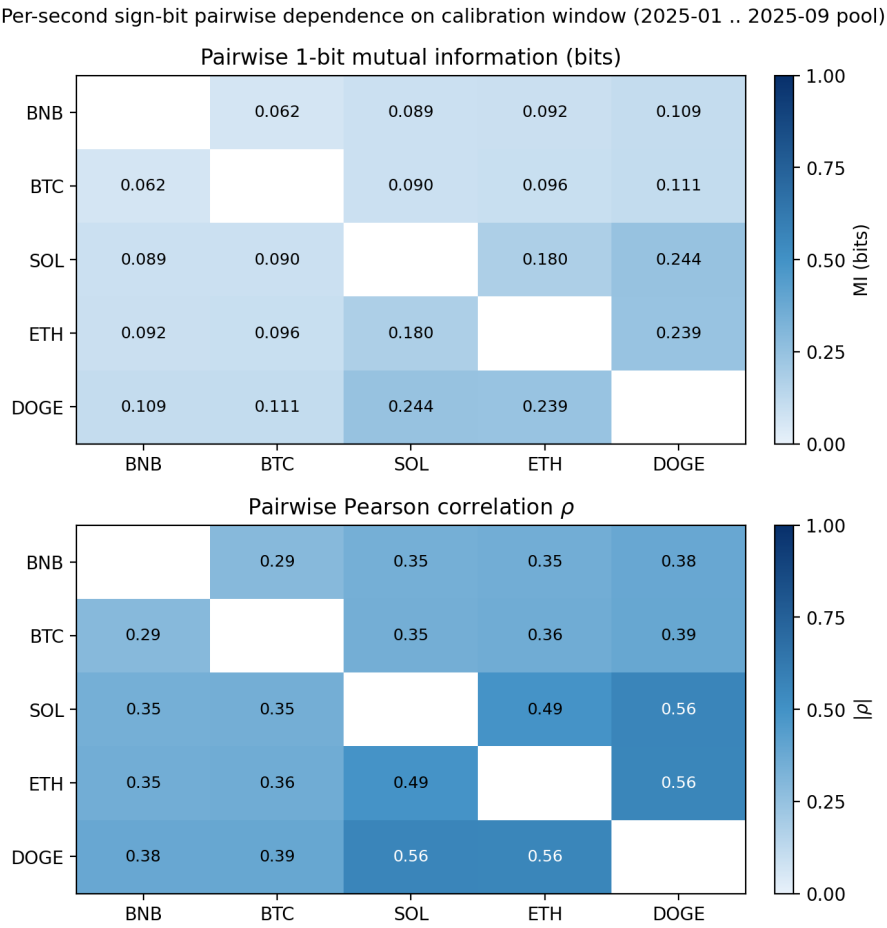

Figure 4.10: Experiment 4 – calibration-window mutual information matrix.

Pairwise mutual information between the five 1-second sign-bit streams over the 2025-01 to 2025-09 calibration window.

Fig. 4.10 first checks the main assumption behind the combination step: whether the asset streams

are close to independent. The answer is no, but the diagnostic is still useful. Using 10^{-3} bits/symbol as a practical near-independence reference level, all 10 asset pairs are above this level. The pooled mutual information ranges from 0.062 bits (BTC–BNB) to 0.244 bits (SOL–DOGE), with a mean of 0.131 bits. The absolute correlation $|\rho|$ ranges from 0.292 to 0.565, with a mean of 0.409.

This means that crypto 1-second signs share a clear common-factor movement. The strict independence assumption behind the piling-up intuition is not satisfied. For this reason, the mutual information (MI) matrix is not used as the subset-selection rule. It is used as a diagnostic for interpreting combined-stream bias: if combination improves the residual tests, the correct reading is "partial whitening from partial independence", not a theorem under independent inputs.

4.5.3 Calibration Results

The calibration procedure is simple. For each subset and each calibration month, the combined stream is built and the $\ell = 1, \ldots, 400$ grid is scanned. The smallest passing ℓ is recorded as that month's ℓ^*. Then, within each n, all subsets are compared by estimated bits/month, and the throughput-best subset is selected.

Table 4.8: Experiment 4 – calibration-selected throughput-best subset.

n	selected subset	ℓ_n^*	selected output offset	est. bits/mo	Δ vs runner-up	median $p(1)$
2	BTC + ETH	10	1	97,163	$+5\%$	0.340
3	BTC + ETH + SOL	3	0	201,950	$+15\%$	0.500
4	ETH + BNB + SOL + DOGE	9	1	45,711	$+8\%$	0.281
5	BTC + ETH + BNB + SOL + DOGE	3	0	102,858	unique	0.499

"Δ vs runner-up" is the throughput gap between the selected subset and the second-best subset with the same n. For $n = 5$ there is only one subset.

Table 4.8 gives three observations. First, the throughput-best subset is not the same as the MI-best subset. For example, the MI-best subset for $n = 3$ is BTC+BNB+SOL, while the throughput-best subset is BTC+ETH+SOL. The main text uses the throughput-best subset because the deployment goal is usable output rate under the fixed criterion. MI remains a diagnostic.

Second, odd and even n behave differently in this calibration window. For $n = 3$ and $n = 5$, the combined $p(1)$ is almost 0.5, and ℓ_n^* is only 3. For the even cases $n = 2$ and $n = 4$, $p(1)$ is further from 0.5, so a larger ℓ is needed to pass Monobit. This odd/even explanation is only an empirical diagnostic, but it matches the symmetry intuition of XOR in this dataset.

Third, the throughput advantage of the selected subset is not always large. The gap is only $+5\%$ for $n = 2$ and $+8\%$ for $n = 4$; it is clearer for $n = 3$ at $+15\%$. Thus, for $n = 2$ and $n = 4$, "throughput-best" is only a marginal calibration advantage. The final deployment set must depend on validation robustness, not only on calibration throughput ranking.

The complete calibration ranking of all 26 subsets is given in Table A.9.

4.5.4 Validation Results

Validation fixes the selected subset, ℓ_n^*, and selected output offset from calibration. The streams are rebuilt month by month from 2025-10 to 2026-03. No parameter is re-selected. The 29-sub-test

verdicts are still based on all-offset pass rates. The selected output offset fixes the output stream and throughput measurement, but it does not decide the validation battery pass/fail result.

Table 4.9: Experiment 4 – validation throughput and failure count.

n	selected subset	ℓ_n^*	output bits/mo	hours/256 bits	sub-tests with ≥ 1 fail / 29
2	BTC + ETH	10	95,622	1.99	4
3	BTC + ETH + SOL	3	191,830	0.98	11
4	ETH + BNB + SOL + DOGE	9	35,583	5.28	18
5	BTC + ETH + BNB + SOL + DOGE	3	95,118	2.01	8

Throughput is measured on the selected output offset fixed during calibration. Failure count is the number of sub-tests that fail in at least one of the six validation months.

The first validation result is that throughput is usable for low-frequency password generation. The $n = 3$ configuration has the highest median throughput, about 191,830 bits/month, or about 0.98 hours for 256 bits. The $n = 2$ and $n = 5$ configurations are both near 95k bits/month, or about 2 hours for 256 bits. The $n = 4$ configuration is slower, about 35,583 bits/month, or 5.28 hours for 256 bits. For deployment latency, $n = 2$, $n = 3$, and $n = 5$ are all in an acceptable range for occasional password generation. Throughput is not the main bottleneck.

Validation throughput is close to, but lower than, the calibration estimate. The largest drop is for $n = 4$ (about 22%), which matches the later diagnosis that this configuration has moved outside the calibration regime.

Table 4.10: Experiment 4 – key validation sub-tests, PASS months out of 6.

Sub-test	$n = 2$	$n = 3$	$n = 4$	$n = 5$
D (adaptive k)	6	6	6	6
$D(k = 2)$	6	4	6	6
Monobit	4	6	0	5
Runs	6	4	6	6
ApEn	6	6	2	6
Block Frequency	6	5	3	6
CumSum forward	4	6	2	5
CumSum backward	4	6	0	5
DFT	6	6	6	6
Serial m	6	5	2	6
Serial $m-1$	6	5	6	6
MultinomialBitsOver L4	6	5	1	6

The second validation result is statistical robustness. The validation results clearly separate deployable configurations from a failure case. The $n = 2$, $n = 3$, and $n = 5$ configurations still have some failed sub-tests, but the failures are local. The $n = 4$ configuration fails systematically.

For $n = 2$, the weakness is concentrated in marginal-bias-sensitive tests: Monobit, CumSum forward/backward, and Alphabit MultinomialBitsOver L2 each fail in 2 of the 6 months. L2 is another

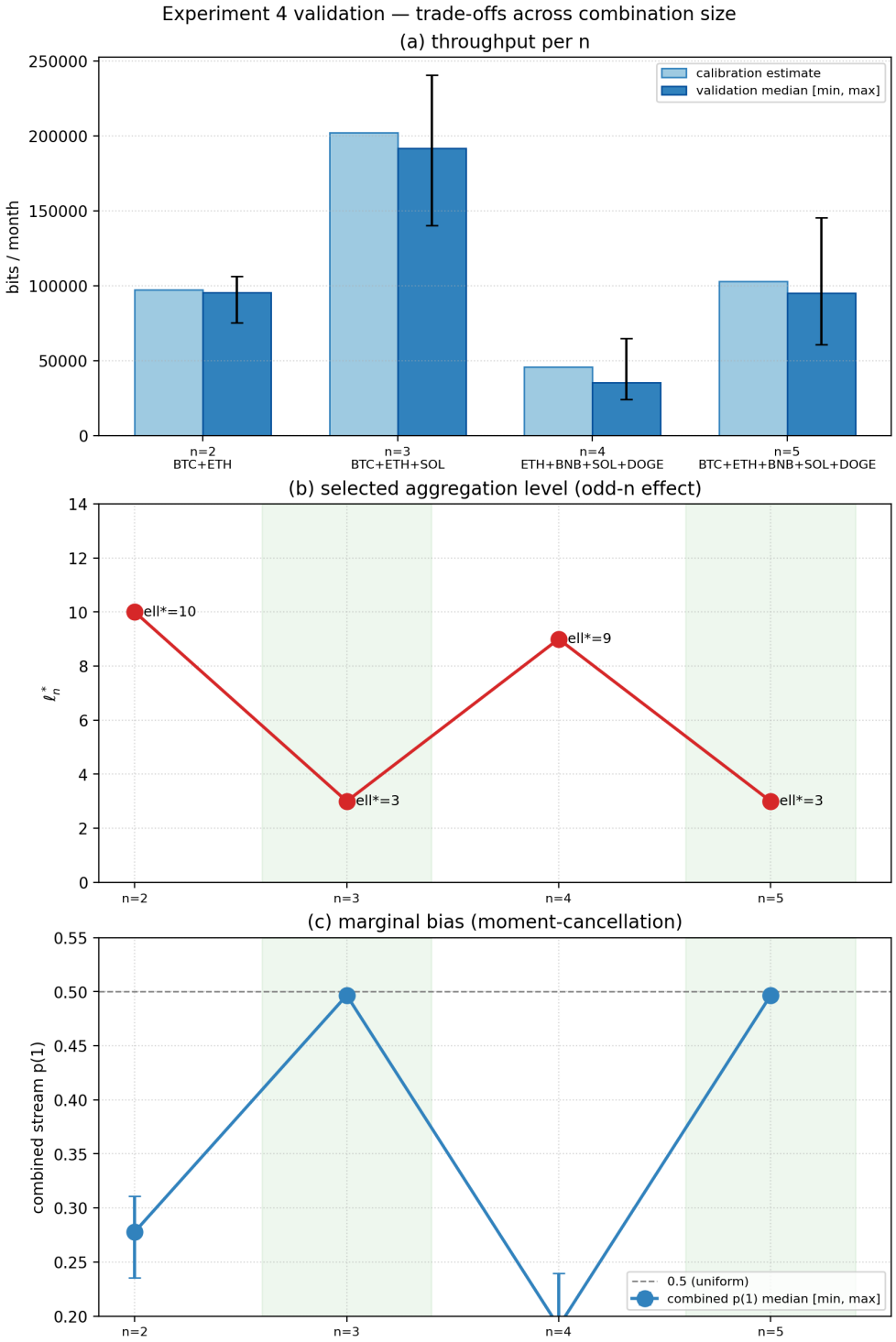

Figure 4.11: Experiment 4 – validation throughput and trade-off summary.

The figure shows validation throughput, combined-stream marginal balance, and the distribution of failures for the four selected configurations.

Alphabit length setting; Table 4.10 lists L4 because L4 is the key comparison with Experiment 3. For $n = 3$, Monobit and CumSum pass all months, but $D(k = 2)$ and Runs each fail in 2 of the 6

months. This suggests that $\ell = 3$ is enough for marginal balance, but a little shallow for short-range dependence. For $n = 5$, the result is the most stable: Serial m and Serial $m-1$ both pass in all 6 months, and most failures appear in only 1 month.

The $n = 4$ case is different. Monobit passes in 0 of 6 months, CumSum backward in 0 of 6, CumSum forward in 2 of 6, Serial m in 2 of 6, and MultinomialBitsOver L4 in 1 of 6. Across the full 29-sub-test universe, 18 sub-tests fail at least once. This is a systematic validation failure, not a small local weakness.

Table 4.10 also closes the problem left by Experiment 3. To avoid comparing different time windows, the single-asset +Runs verdicts from Experiment 3 are restricted to the same validation window (2025-10 to 2026-03). In that window, single-asset +Runs streams pass Serial m and MultinomialBitsOver L4 in only 10 of 22 admissible cells. By contrast, the deployable combined streams reach Serial m pass counts of $6/6$, $5/6$, and $6/6$ for $n = 2, 3, 5$, and MultinomialBitsOver L4 pass counts of $6/6$, $5/6$, and $6/6$.

This paired comparison shows that multi-asset combination improves the higher-order residuals found in Experiment 3. The exception is $n = 4$: Serial m passes only $2/6$ and MultinomialBitsOver L4 only $1/6$. The reason is that its marginal bias has moved outside the calibration range. In short, multi-asset combination helps, but only when the fixed calibration remains representative of validation.

$n = 4$ **is retained as a negative validation result.** The $n = 4$ case fully passes calibration. Across the five possible 4-asset subsets and nine calibration months, every cell can find an ℓ^* inside the $\ell = 1, \ldots, 400$ grid that passes the +Runs criterion. ETH+BNB+SOL+DOGE is selected because it has the highest estimated throughput. In validation, however, this same fixed configuration fails all six months on Monobit and CumSum backward. The failure is a generalisation failure caused by non-stationary sign bias.

The failure can be explained by drift in the marginal bias of the combined stream. During calibration, the $n = 4$ combined $p(1)$ is around 0.27, with monthly values from about 0.237 to 0.303. During validation, it moves down to about 0.15–0.24, and in five of the six months it is more biased than any calibration month. The fixed $\ell = 9$ selected in calibration is calibrated for the $p \approx 0.27$ range. At that bias level, 9-bit XOR aggregation is enough to push the residual bias below the Monobit detection level. When p moves close to 0.15, the input bias becomes larger, and the same $\ell = 9$ is no longer enough. Monobit therefore fails throughout validation.

This is the value of the 9/6 split. It does not only give a failed configuration; it exposes a boundary of the method. Under month-scale non-stationarity in crypto sign bits, a fixed ℓ selected in calibration cannot be assumed to generalise for every n. The $n = 4$ configuration is therefore excluded from the deployment set, but it is kept in the thesis as a negative result.

4.5.5 Min-Entropy and Deployment Set

The validation result leads to a deployment set, not to one single winner. The $n = 3$ configuration has the highest throughput. The $n = 5$ configuration is the stronger robustness check and has a similar hours-per-256-bit value to $n = 2$. The $n = 2$ configuration is kept as a lower-complexity fallback. The $n = 4$ configuration is excluded because its validation failure is systematic.

Using the min-entropy budget from § 3.7, each validation cell is evaluated with the MCV and Markov estimators, and the smaller value is used as h_∞.

Table 4.11 agrees with the validation battery. If all n values are included, the worst cell is $n = 4$ with $h_\infty = 0.903$ bits/symbol, requiring 36 input bytes for a 256-bit target. If the deployment set

Table 4.11: Experiment 4 – validation min-entropy summary.

n	selected subset	median h_∞	worst h_∞	binding estimator	IKM bytes
2	BTC + ETH	0.979	0.975	MCV	33
3	BTC + ETH + SOL	0.988	0.982	mostly MCV; worst Markov	33
4	ETH + BNB + SOL + DOGE	0.943	0.903	MCV	36
5	BTC + ETH + BNB + SOL + DOGE	0.982	0.979	MCV	33

h_∞ is the smaller of the MCV and Markov estimates for each validation cell. The IKM byte count is computed against a 256-bit target.

is restricted to $n \in \{2, 3, 5\}$, the worst-month value is $h_\infty = 0.975$ bits/symbol, so 33 input bytes are enough for the 256-bit target. MCV is almost always the binding estimator: among 24 validation cells, 23 are bound by MCV and only 1 by Markov.

This means that the main residual is usually 0/1 imbalance, which MCV captures, rather than adjacent-bit dependence, which Markov captures. The combined streams are therefore positioned as entropy input for downstream conditioning.

Experiment 4 gives the core experimental evidence for RQ3. Multi-asset XOR combination can produce validation-window streams with usable throughput and enough estimated min-entropy for password generation, as long as the failed $n = 4$ configuration is excluded and HKDF-based conditioning is treated as a required part of the design. The conclusion is positive but conditional: combination improves the high-order residual seen in Experiment 3 for the deployable configurations, while $n = 4$ exposes the non-stationarity boundary of fixed calibration.

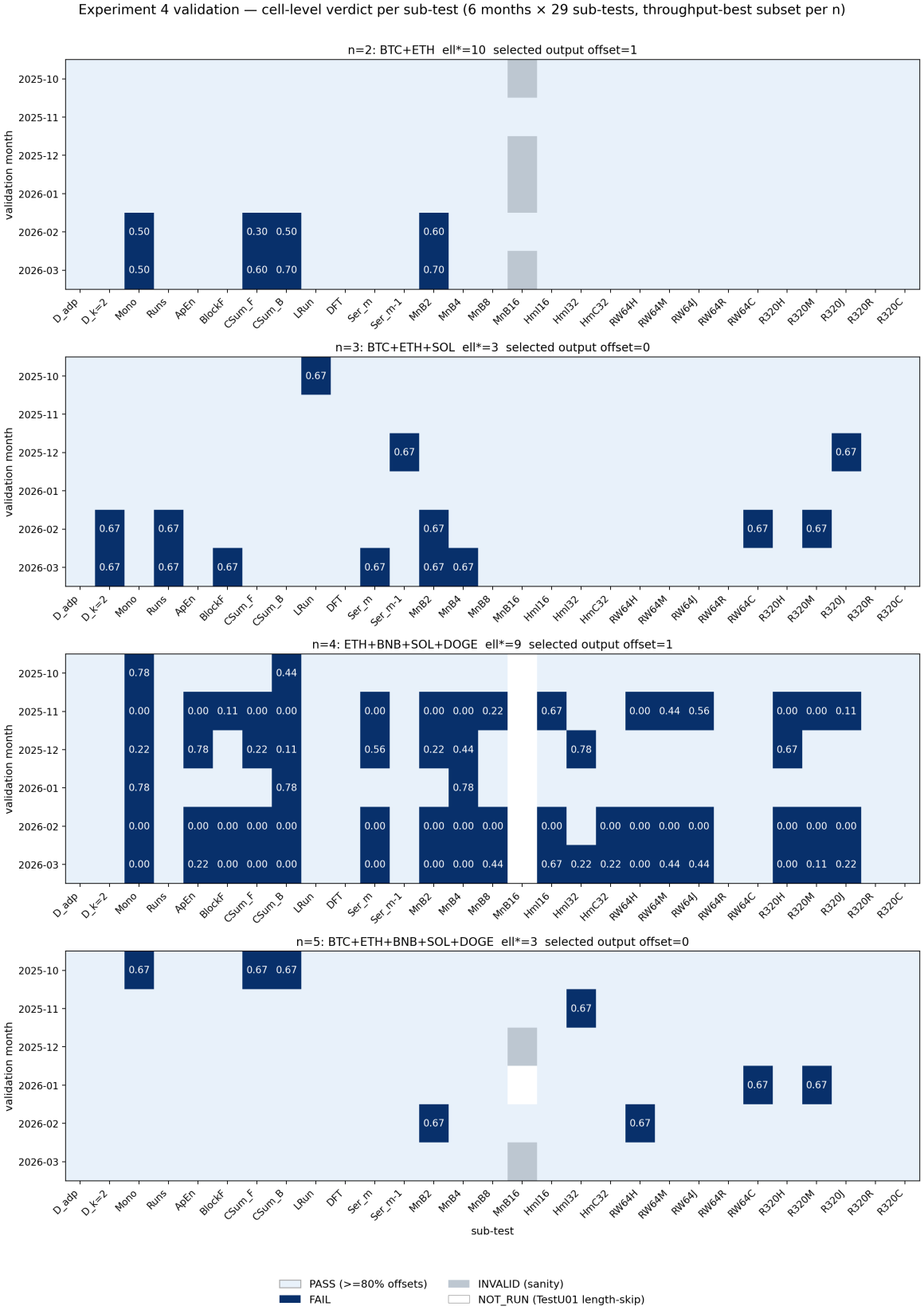

Figure 4.12: Experiment 4 – validation verdict matrix.

The matrix shows pass/fail verdicts for the 29-sub-test universe over the six validation months.

Chapter 5

Password Generator Prototype

This chapter connects the deployable streams selected in Experiment 4 to a standard HKDF-SHA256 conditioning and character-mapping pipeline. The output is a set of 16-character passwords, and the evaluation checks whether their output-level statistics match an ideal baseline.

This chapter is an application-layer validation. The statistical randomness and min-entropy of the market input have already been evaluated in Chapter 4; the question here is whether streams that pass the previous screening can be turned by standard cryptographic conditioning into usable passwords, and what strength those passwords have under the output metrics used here.

5.1 Prototype Design and Scope

The prototype uses the deployable set from Experiment 4: $n \in \{2, 3, 5\}$. The $n = 4$ configuration is excluded because it fails systematically in validation. The goal is to check whether these streams, after HKDF-SHA256 conditioning and character mapping, can produce passwords that are strong in the everyday random-password sense.

In this chapter, a "strong password" means a randomly generated password string with a near-uniform character distribution and search-space entropy clearly above common human-chosen passwords. The main strength measure is the password length and the character set size, as discussed in § 2.2.

Fig. 5.1 shows the per-password pipeline. Each password uses a disjoint 33-byte IKM block cut from an Experiment 4 deployable stream. The block is passed through HKDF-SHA256, then mapped into a 70-character alphabet:

```
[a-z] + [A-Z] + [0-9] + !@#$%^&*.
```

The 33-byte IKM size follows the min-entropy estimate in Table 4.11: for the deployable set, the worst validation month has $h_\infty = 0.975$ bits/symbol, so $33 \times 8 \times 0.975 \approx 257$ bits of estimated input entropy, slightly above a 256-bit target.

Rejection sampling is used to avoid modulo bias (Lemire, 2019). The accepted byte range is $b < 210$, because 210 is the largest multiple of 70 below 256. The HKDF implementation is checked against RFC 5869 Appendix A Test Case 1 (Krawczyk and Eronen, 2010). In this run, all 2,400 HKDF-generated passwords finish in one expand round, and the re-expand fallback is never used.

This chapter uses output-level character uniformity and Shannon entropy rather than re-running

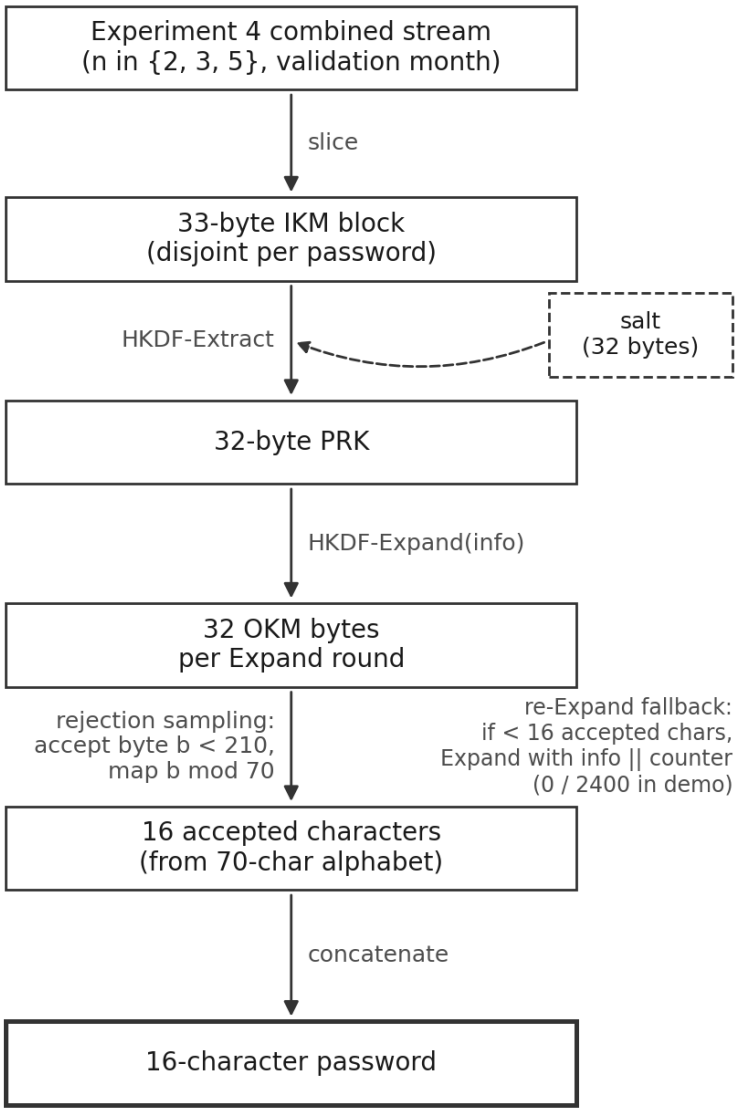

Figure 5.1: Prototype pipeline from Experiment 4 combined stream to password output.

Each password uses a disjoint 33-byte IKM block from a deployable Experiment 4 stream. HKDF-SHA256 conditions the input, and rejection sampling maps the output bytes into a 70-character password alphabet.

the full NIST/TestU01 battery on the password strings, since the statistical randomness of the market input has already been evaluated in Chapter 4.

The prototype uses a repository-persisted salt so that the demo output can be reproduced exactly. This salt is suitable for reproducing the experiment: the same market streams, code, and salt generate the same passwords. Since market data are public, a production password generator would need to combine the market-derived entropy with non-public deployment-secret material during conditioning. The public demo salt in the repository is not suitable as the only secret source in a production setting.

5.2 Evaluation and Results

The evaluation uses the same six validation months as Experiment 4, from 2025-10 to 2026-03. The market group uses the deployable configurations $n = 2$, $n = 3$, and $n = 5$. For each (n, month) cell, the prototype generates 100 passwords of length 16, so the market group contains $3 \times 6 \times 100 = 1800$ passwords.

Two baselines are generated for comparison. The first is B1, an ideal-password baseline: passwords are sampled uniformly from the same 70-character alphabet without HKDF. This is the ideal output-level distribution. The second is B2, an ideal-IKM baseline: random IKM is passed through the same HKDF-SHA256 and character-mapping pipeline. This keeps the conditioning and mapping steps, but replaces the entropy source with ideal input. Each baseline has 6 monthly cells and 100 passwords per cell. The total evaluation size is therefore 3000 passwords across 30 cells.

Each cell is evaluated with two output metrics. First, the 100 passwords are expanded into 1600 characters, and a pooled chi-square goodness-of-fit test (Pearson, 1900) checks whether the 70 character categories are close to uniform at $\alpha = 0.01$. Second, Shannon entropy is computed on the same 1600 characters, in bits per character.

Table 5.1: Prototype output evaluation.

Group	cells	chi-square pass	chi-square p-value range	entropy median [min, max]
market $n = 2$	6	6/6	$[0.248,\ 0.897]$	$6.1000\ [6.0946,\ 6.1045]$
market $n = 3$	6	6/6	$[0.111,\ 0.783]$	$6.0986\ [6.0902,\ 6.1032]$
market $n = 5$	6	6/6	$[0.0705,\ 0.982]$	$6.0950\ [6.0897,\ 6.1082]$
B1 ideal-password	6	5/6	$[0.000375,\ 0.787]$	$6.0981\ [6.0779,\ 6.1017]$
B2 ideal-IKM	6	6/6	$[0.595,\ 0.938]$	$6.1007\ [6.0996,\ 6.1058]$

The market-derived passwords pass the output uniformity test. All 18 market-derived cells pass the chi-square test. Across all 30 cells, only one cell fails, and it is in the B1 ideal-password baseline for 2026-01 ($p = 0.000375$), not in the market output. With $\alpha = 0.01$ and 30 cells, the expected number of type-I failures is $30 \times 0.01 = 0.30$, and the probability of at least one failure is about 26%. This single baseline failure is therefore best read as normal finite-sample variation, not as a problem in the prototype pipeline.

The empirical entropy matches the two baselines. The theoretical maximum is $\log_2(70) = 6.1293$ bits/char. The median entropy of all groups lies around 6.095–6.101 bits/char. The observed value is about 0.02–0.05 bits/char below the theoretical maximum, but the same gap appears in the market groups and in both baselines. The main reason is the finite-sample bias of the plug-in entropy estimator on 1600 characters. The Miller–Madow term is approximately $(m-1)/(2N \ln 2) \approx 0.031$ bits/char for $m = 70$ and $N = 1600$ (Miller, 1955); see also Paninski (2003).

The generated passwords fall in the strong-password range. Using the overall median of 6.0997 bits/char, a 16-character password has about 97.6 bits of empirical entropy. The theoretical search-space upper bound is $16 \times \log_2(70) \approx 98.07$ bits. This is far above the common scale of human-chosen passwords and above the 80-bit working benchmark discussed in § 2.2.

The three deployable n values look similar at the output layer. Market $n = 2$, $n = 3$, and $n = 5$ fall in the same output range. Under the min-entropy conditions established in § 4.5.5, their upstream

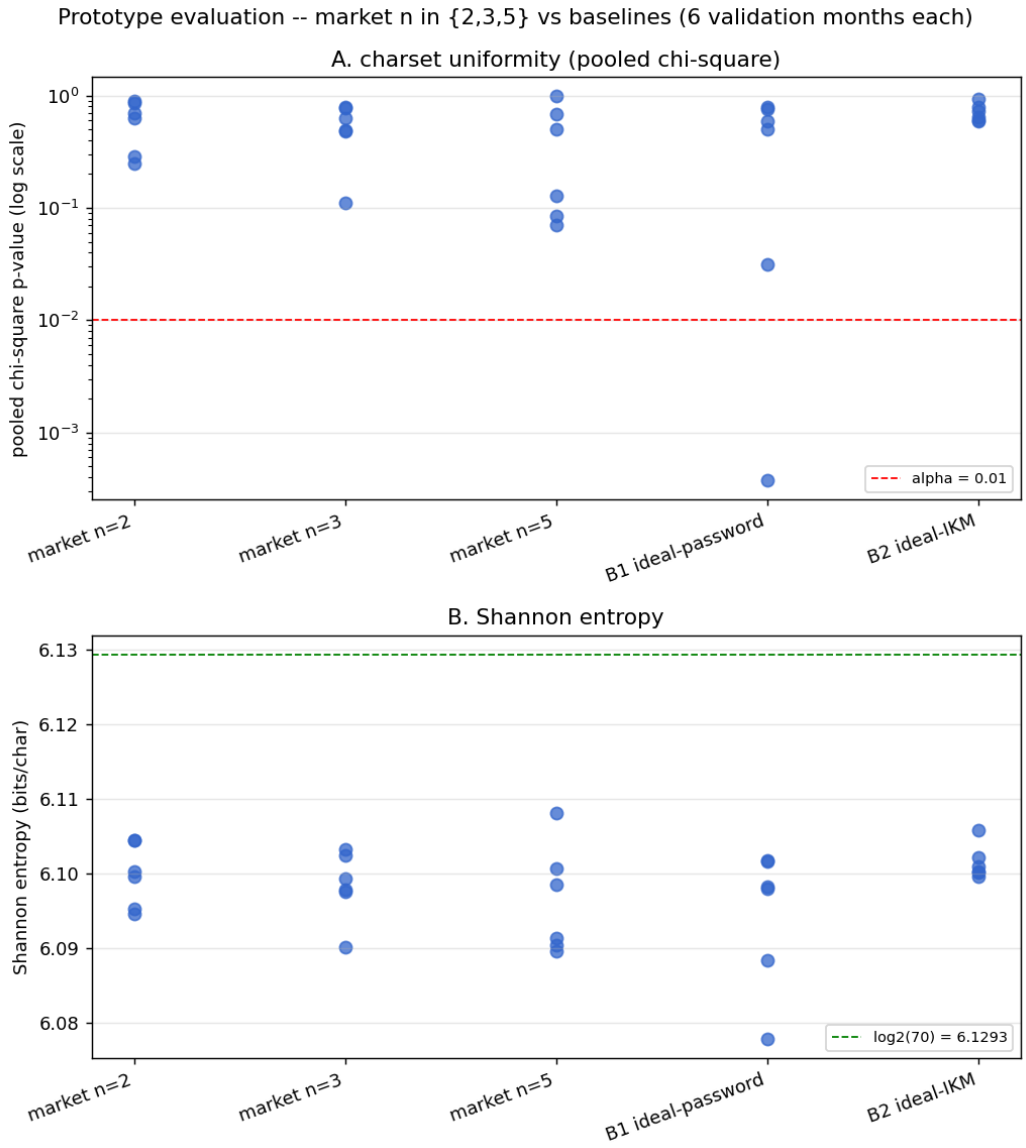

Figure 5.2: Prototype evaluation – chi-square and Shannon entropy.

Market outputs for $n \in \{2, 3, 5\}$ are compared with the ideal-password baseline B1 and the ideal-IKM baseline B2.

residuals do not show up as extra abnormalities in the character-uniformity and Shannon-entropy metrics used in this chapter.

Overall, Chapter 5 gives a positive application result. After the screening, aggregation, extended-battery validation, multi-asset combination, and min-entropy sizing in Experiments 1–4, the deployable market-derived streams can be used as entropy input for an HKDF-based password generator. The output passwords match the ideal baselines on the metrics used here and fall in the everyday strong-password range.

Chapter 6

Discussion

6.1 Thesis-Level Answer

The thesis-level question answered by this work is: in a passive statistical setting, can public cryptocurrency market data, after statistical aggregation and cryptographic conditioning, provide usable entropy for strong password generation?

The central answer is a conditional yes: for the deployable configurations $n \in \{2, 3, 5\}$, cryptocurrency market data can be used as entropy input for HKDF-SHA256 conditioning, and the resulting prototype produces about 98-bit random passwords whose output statistics match the ideal baselines used in Chapter 5.

This answer is based on the whole chain of experiments, from the raw baseline through one-second-bar aggregation, the extended test battery, and multi-asset combination, to the password prototype.

6.2 Answers to Research Questions

RQ1 – Can aggregation extract statistically independent random sequences from cryptocurrency price data? The answer is a conditional yes within the statistical-test framework of this thesis. Both aggregation axes can find an ℓ^* that passes D + Monobit on most (asset, month) cells: transaction-time strict criterion gives $63/75$, transaction-time relaxed criterion gives $75/75$, and 1-second bar strict criterion gives $70/75$. However, the independence evidence differs strongly between the two axes. On the transaction-time axis, $D(k = 2)$ and Runs still reject almost every selected output offset stream. On the 1-second bar axis, the +Runs criterion reaches $62/75$. The stronger independence result therefore comes from the physical-time axis, and this is why Experiments 3 and 4 use the 1-second bar +Runs sample.

RQ2 – Can the sequences pass standard randomness tests? The answer is mostly yes, but with a clear boundary. For the 62 cells selected by the 1-second bar +Runs criterion, Experiment 3 shows that D (adaptive k), Monobit, Runs, and DFT pass in all admissible cells, and $D(k = 2)$ fails in only one of the 62 cells. At the same time, Serial m, TestU01 Alphabit MultinomialBitsOver L4, and ApEn still expose higher-order residuals across assets. The small criterion is effective for marginal and short-range structure, but it does not cover higher-order and multi-symbol structure. This boundary is the direct reason for adding multi-asset combination in Experiment 4.

RQ3 – Can the sequences be used for secure password generation? The answer is yes for the deployable configurations evaluated here. Experiment 4 selects the deployable set $n \in \{2, 3, 5\}$ by a 9/6 calibration-validation split. The $n = 4$ configuration fails systematically in validation, with Monobit and CumSum backward passing $0/6$, so it is kept as a negative result and excluded from deployment. Chapter 5 then turns the deployable streams into passwords. All the market-derived cells pass the character-uniformity test, and their Shannon entropy matches the ideal baselines (see Table 5.1). This supports the claim that standard cryptographic conditioning is a necessary part of turning the deployable streams into usable passwords.

6.3 Secondary Observations

Beyond the three research questions, two secondary observations are worth recording. They connect the cryptocurrency results back to the high-frequency-finance literature reviewed in Chapter 2, but they are not counted as the main contribution.

Coarse activity layering with local non-monotonicity. The selected ℓ^* values show a coarse two-layer structure across the five assets. High-activity BTC usually needs a larger ℓ^*, which is in the same direction as earlier equity-market results where higher activity requires deeper aggregation. At the finer level, however, SOL, DOGE, and BNB are not strictly monotone: raw trades/s ranking and selected ℓ^* ranking do not fully match. Similar non-monotonic behaviour has been observed in other markets and is usually linked to microstructure mechanisms such as order splitting and execution scheduling (Almgren and Chriss, 2001). Identifying the exact mechanism in cryptocurrency data is left to future work.

First-order residuals are partly an axis artifact. The RQ1 answer (§ 6.2) shows that $D(k = 2)$ and Runs are severe on the transaction-time axis but weak on the 1-second bar axis. The point worth recording here is how to read that result: what looks like a fixed property of the source data is, in part, a property of the sampling scheme rather than of the market itself.

6.4 Limitations and Future Work

Methodological limitations.

1. **The selected output offset rule is still simple.** This thesis fixes the selected output offset in the last calibration month and keeps it fixed in validation. This affects the final output stream and the throughput measurement, but not the validation battery verdict, which is still based on all-offset pass rates. A stronger deployment version should compare this rule with single-month selection, multi-month stability-based offset selection, and a fixed-offset rule.

2. **The criterion design is not formally corrected over the whole search.** The ℓ grid search is not corrected with Bonferroni or FDR. The relaxed criterion parameters $(F, f) = (3, 0.03)$ are a design choice from § 3.4, not an optimum over a parameter family. Other values, such as $(2, 0.02)$ or $(5, 0.05)$, may change coverage and selected ℓ^*, and this sensitivity is not explored here.

3. **The extended battery is still not the full NIST/TestU01 universe.** Experiment 3 adds 7 NIST SP800-22 extensions and TestU01 Alphabit, but it does not include Non-overlapping Template, Universal Statistical, the Random Excursions family, or TestU01 Rabbit. This is a scope and length limitation, not a claim that these tests are unimportant.

4. **Some accepted offset streams are near the lower length boundary.** The valid offset streams in this thesis are often in the range from 2×10^3 to about 10^4 bits. Shternshis and Marmi's Appendix A simulations suggest that the χ^2 approximation for the D test is more reliable around $n \geq 10^4$ and can deviate around $n \sim 10^3$. Near-boundary verdicts should therefore be read with care. Longer sample windows would reduce this risk.

5. **Active market manipulation is outside the threat model.** The prototype is evaluated under the passive statistical setting described above and in § 5.1. If the target becomes a public randomness beacon or an on-chain protocol, the work would need a separate active-attacker and cost-of-manipulation model, closer to Landis and Bonneau (2025).

Future work.

- **Complete the remaining batteries.** Future work should add the omitted NIST SP800-22 tests and TestU01 Rabbit where the bit lengths allow it, and report length-skip and admissible denominators consistently.
- **Extend the sample and asset universe.** A longer time period, more assets, derivatives markets, and cross-exchange data would test whether the selected ℓ^* patterns and the $n = 4$ failure are stable or market-specific.
- **Test compression-based complexity diagnostics.** Lempel–Ziv style methods (Ziv and Lempel, 1977) and compression-based language-tree methods (Benedetto et al., 2002) could be used as additional randomness diagnostics on longer bit streams. They were not used in the main analysis because the current cell lengths and ℓ ranges make their power limited.
- **Explore low-frequency key-material use.** The same 33-byte IKM to HKDF-Extract pipeline could be studied for low-frequency key-material derivation. This direction belongs to a new key-material setting and would require a new threat model, secret-management design, and authentication requirements.

Chapter 7

Conclusion

7.1 Recap of the Motivation and Setting

This thesis starts from a concrete engineering question: in a passive statistical setting, can public cryptocurrency market data, after statistical aggregation and cryptographic conditioning, provide usable entropy for strong password generation? What matters is not only producing bits, but whether the bits pass standard randomness tests, whether the throughput is usable, and whether, after cryptographic conditioning, they can support the final password output.

Experiment 1 shows that raw tick signs are not suitable. They have high marginal entropy, but they are conditionally predictable. This makes aggregation necessary. Experiments 2–4 and the Chapter 5 prototype are built on that result.

The method is based on two earlier ideas: the entropy-based predictability test D from Shternshis and Marmi (2025), and the all-offset construction from Onofri et al. (2025). This thesis adapts these ideas to the 24/7 cryptocurrency setting by adding a relaxed criterion, a 1-second bar physical-time pipeline, multi-asset XOR combination, and output metrics for throughput and min-entropy.

The final answer is yes, but with conditions. For the deployable configurations $n \in \{2, 3, 5\}$, cryptocurrency market data can be used as entropy input for HKDF-SHA256 conditioning, and the prototype can produce about 98-bit random passwords whose output-level statistics match ideal baselines. This result holds within the passive statistical setting of this thesis; an active-attacker setting or production deployment would need a separate analysis.

7.2 Summary of Contributions

The contributions of this thesis can be grouped into two levels: method-level and application.

Method-level contributions. The thesis combines the D test, the all-offset construction, and classical NIST SP800-22 / TestU01 sub-tests into one all-offset acceptance and audit framework. It then evaluates this framework on the 24/7 cryptocurrency market. On top of the reused components, the thesis adds four concrete method-level parts:

- **Relaxed criterion** (§ 3.4): a heuristic robustness check that keeps offset redundancy without using Bonferroni / Šidák as a false-pass correction.

- **1-second bar physical-time pipeline** (§ 3.5): a second aggregation axis that reduces short-range dependence more clearly than the transaction-time axis.
- **Multi-asset XOR combination** (§ 3.6): a bit-domain aggregation method that is explicitly separated from the price-domain aggregation used earlier.
- **Output metrics** (§ 3.7): a common reporting frame for throughput and a simplified SP800-90B-style min-entropy estimate.

Application contribution. Chapter 5 gives an end-to-end password generator prototype. It connects the Experiment 4 deployable streams to HKDF-SHA256 conditioning and 70-character rejection sampling. In the evaluation, all the market-derived cells pass the character-uniformity test and their Shannon entropy matches the ideal baselines (see Table 5.1). This is the application-level evidence that standard cryptographic conditioning can turn the deployable streams into usable passwords.

Overall, the thesis shows that market-derived randomness is not usable in raw form, but it can become useful after aggregation, extended testing, multi-asset combination, min-entropy sizing, and cryptographic conditioning.

The implementation code, experiment scripts, and processed outputs are available in the project repository: https://github.com/Liu-yujia-66/CryptoEntropy-RNG.

Appendix A

Supplementary Tables

This appendix gives two kinds of supplementary results. First, it gives the full 15-month tables of selected ℓ^* for the criteria in Experiment 2 (§ 4.3). The figures in the main text show only two quarters (2025 Q1 and 2026 Q1, i.e. Q5); the full month-by-month results are listed here. Second, it gives the complete calibration ranking for Experiment 4 (§ 4.5). In the Experiment 2 tables, all tables use the same asset order (BTC, ETH, BNB, SOL, DOGE), which is the same descending order of raw trades/s as in Table 4.1. An empty cell ("–") means that the (asset, month) cell has no acceptable ℓ inside the ℓ grid under the given criterion. The raw CSV files behind the Experiment 2 appendix tables live under `data/processed/experiment2/`; the Experiment 4 calibration summary is under `data/processed/experiment4/calibration_all_subsets/`.

Table A.1: Single-offset criterion – selected ℓ^* per (asset, month). The ℓ grid is 50 to 2000 with step 25; the criterion is the smallest ℓ at which D (adaptive k) and Monobit both give $p \geq \alpha$ ($\alpha = 0.01$).

Month	BTC	ETH	BNB	SOL	DOGE
2025.01	–	925	275	750	825
2025.02	1650	875	475	500	225
2025.03	1800	625	250	275	375
2025.04	1700	750	250	225	275
2025.05	1500	1250	250	400	225
2025.06	1100	525	200	225	125
2025.07	1975	600	375	400	250
2025.08	–	1575	275	475	250
2025.09	900	850	350	300	250
2025.10	1375	725	850	225	175
2025.11	1125	875	300	150	150
2025.12	1050	600	275	125	75
2026.01	1300	725	250	175	125
2026.02	1225	550	250	200	100
2026.03	1075	575	175	100	100

Table A.2: All-offset strict criterion – selected ℓ^* per (asset, month). The ℓ grid is 50 to 2000 with step 25; the criterion is the 80% strict criterion (pass rates of D and Monobit both ≥ 0.80).

Month	BTC	ETH	BNB	SOL	DOGE
2025.01	–	1900	325	–	1075
2025.02	–	1025	700	–	900
2025.03	–	1150	350	425	500
2025.04	–	975	275	600	375
2025.05	–	–	475	625	650
2025.06	1325	850	250	300	125
2025.07	–	1275	425	550	650
2025.08	–	–	300	650	375
2025.09	1500	975	375	425	450
2025.10	1625	1300	1025	350	225
2025.11	1600	1400	450	225	200
2025.12	1275	700	275	150	75
2026.01	1825	950	400	225	150
2026.02	–	650	325	225	125
2026.03	1225	750	200	150	125

Overall acceptance rate is $63/75$ (BTC fails on 8 months, ETH and SOL each fail on 2 months). On these 63 accepted cells, the Runs pass rate is below 0.05 on 38 cells, and the $D(k{=}2)$ pass rate is below 0.05 on the same 38 cells.

Table A.3: Transaction-time strict + Runs criterion – selected ℓ^* per (asset, month). The ℓ grid is the same as Table A.2; the criterion adds Runs to the strict criterion, that is, pass rates of D, Monobit and Runs are all ≥ 0.80. This table is the same-criterion comparator referenced in § 4.3.5; it does not change the strict-criterion definition in the main text.

Month	BTC	ETH	BNB	SOL	DOGE
2025.01	–	–	675	–	–
2025.02	–	1800	–	–	1500
2025.03	–	1550	550	1300	950
2025.04	–	1575	425	950	1075
2025.05	–	–	1075	1200	1550
2025.06	–	1325	550	675	225
2025.07	–	–	–	1650	–
2025.08	–	–	500	1750	1075
2025.09	–	1725	650	750	1025
2025.10	–	–	–	550	300
2025.11	–	1850	525	325	1425
2025.12	1925	1050	400	250	100
2026.01	–	1850	650	350	275
2026.02	–	–	575	450	300
2026.03	–	1750	225	325	300

Overall acceptance rate is $48/75$ (BTC passes on 1 month, ETH on 9, BNB on 12, SOL and DOGE on 13 each).

Table A.4: All-offset relaxed criterion – selected ℓ^* per (asset, month). The ℓ grid is 10 to 2000 with step 2; the criterion is the heuristic relaxed criterion with $(F, f) = (3, 0.03)$.

Month	BTC	ETH	BNB	SOL	DOGE
2025.01	1750	800	244	596	594
2025.02	1550	610	286	312	172
2025.03	1576	436	180	180	214
2025.04	1370	444	176	196	162
2025.05	1248	550	198	226	156
2025.06	924	400	146	104	72
2025.07	1376	576	268	298	220
2025.08	1314	1194	212	388	214
2025.09	900	698	256	270	238
2025.10	1150	714	464	164	124
2025.11	988	660	232	108	72
2025.12	870	470	206	98	48
2026.01	1030	556	218	110	66
2026.02	998	420	158	76	56
2026.03	880	442	128	66	48

Overall acceptance rate is $75/75$.

Table A.5: Bonferroni / Šidák directional comparison on 2025 Q1 (see § 4.3.4). After tightening to per-offset α/N_{valid}, the "at least one offset passes" rule gives selected ℓ^* that are systematically smaller than the values in Table A.4, confirming the directional analysis in § 3.4: Bonferroni / Šidák controls the "at least one false rejection" family-wise error and is not the right correction for the opposite question ("at least one false acceptance"); under an "at least one offset passes" rule, tightening α instead gives an earlier and looser selected ℓ^*. This table is therefore only a directional comparator, not a main criterion.

Month	BTC	ETH	BNB	SOL	DOGE
2025.01	1364	500	218	334	274
2025.02	1144	422	190	208	132
2025.03	1252	338	156	154	102

Table A.6: 1-second bar criterion, base (D + Monobit) – selected ℓ^* per (asset, month). The ℓ unit is seconds; the grid is 10 to 600 with step 1.

Month	BTC	ETH	BNB	SOL	DOGE
2025.01	53	58	148	32	31
2025.02	88	204	61	65	84
2025.03	65	413	70	41	35
2025.04	45	15	57	19	21
2025.05	75	537	327	200	92
2025.06	49	–	111	18	18
2025.07	114	91	–	319	43
2025.08	110	35	111	54	43
2025.09	80	71	254	67	61
2025.10	62	34	38	28	16
2025.11	26	29	25	10	22
2025.12	34	–	91	467	24
2026.01	47	36	76	20	15
2026.02	18	–	27	347	–
2026.03	250	327	197	356	196

Acceptance rate 70/75. Failed cells: 2025.06 ETH, 2025.07 BNB, 2025.12 ETH, 2026.02 DOGE and 2026.02 ETH. Among these, the 2025.12 ETH and 2026.02 DOGE cells have a seconds-with-trades coverage below 0.75; the other failed cells still have a coverage between 0.76 and 0.83.

Table A.7: 1-second bar criterion, +Runs – selected ℓ^* per (asset, month). Same axis, grid and asset order as Table A.6.

Month	BTC	ETH	BNB	SOL	DOGE
2025.01	121	58	148	154	34
2025.02	94	227	77	479	84
2025.03	65	–	70	45	53
2025.04	64	20	112	26	27
2025.05	92	–	327	200	92
2025.06	49	–	111	18	19
2025.07	233	–	–	319	43
2025.08	141	53	111	79	58
2025.09	110	71	254	72	69
2025.10	72	36	39	35	23
2025.11	34	99	27	13	145
2025.12	500	–	91	–	–
2026.01	47	50	76	24	16
2026.02	22	–	27	–	–
2026.03	339	–	293	–	278

Acceptance rate 62/75. Runs passes on most cells here, while on the transaction-time axis it is rejected on almost every cell at the selected ℓ^* (Tables A.2 and A.3); this is the key independence result of the 1-second bar axis (§ 4.3.5).

Table A.8: 1-second bar criterion, +Runs + ApEn ($m = 5$) – selected ℓ^* per (asset, month). Same axis, grid and asset order as Table A.6.

Month	BTC	ETH	BNB	SOL	DOGE
2025.01	121	115	158	154	38
2025.02	94	227	78	479	89
2025.03	70	–	160	46	59
2025.04	64	20	112	26	27
2025.05	94	–	327	200	92
2025.06	51	–	111	18	19
2025.07	233	–	–	319	43
2025.08	141	58	111	79	58
2025.09	110	80	254	73	69
2025.10	72	41	47	35	23
2025.11	34	105	28	15	145
2025.12	500	–	91	–	–
2026.01	49	384	76	24	18
2026.02	22	–	323	–	–
2026.03	339	–	293	–	278

Acceptance rate $62/75$. Adding ApEn at $m = 5$ on top of +Runs does not further reduce the acceptance count; the few cells that move only shift their selected ℓ^* slightly higher.

Table A.9: Experiment 4 per-subset calibration summary.

n	subset	ℓ_n^*	median $p(1)$	est. bits/mo	selected output offset
2	BTC + ETH†	10	0.340	97,163	1
2	ETH + BNB	9	0.333	92,532	0
2	ETH + SOL	12	0.254	80,889	4
2	BTC + SOL	10	0.328	80,888	5
2	ETH + DOGE	13	0.213	79,650	8
2	BNB + SOL	9	0.336	78,314	6
2	BNB + DOGE	10	0.326	75,467	9
2	BTC + BNB	10	0.362	70,219	6
2	BTC + DOGE	12	0.312	67,107	1
2	SOL + DOGE	15	0.207	57,429	2
3	BTC + ETH + SOL†	3	0.500	201,950	0
3	ETH + BNB + SOL	3	0.500	175,462	1
3	BTC + ETH + BNB	3	0.500	174,497	0
3	BNB + SOL + DOGE	3	0.500	166,881	0
3	BTC + BNB + SOL	3	0.499	156,022	1
3	BTC + BNB + DOGE	3	0.500	152,526	1
3	ETH + SOL + DOGE	5	0.499	139,510	0
3	BTC + ETH + DOGE	5	0.500	124,634	4
3	ETH + BNB + DOGE	5	0.502	115,206	2
3	BTC + SOL + DOGE	5	0.499	109,732	2
4	ETH + BNB + SOL + DOGE†	9	0.281	45,711	1
4	BTC + ETH + SOL + DOGE	11	0.273	42,430	5
4	BTC + ETH + BNB + SOL	9	0.320	42,236	1
4	BTC + ETH + BNB + DOGE	10	0.307	38,895	4
4	BTC + BNB + SOL + DOGE	9	0.310	38,511	1
5	BTC + ETH + BNB + SOL + DOGE†	3	0.499	102,858	0

The calibration window is 2025-01 to 2025-09. All 234 (subset, month) cells pass the +Runs criterion, meaning that each cell can find an ℓ^* in the $\ell = 1, \ldots, 400$ grid where D (adaptive k), Monobit, and Runs all reach all-offset pass rate at least 0.80. Rows are ordered by n and then by estimated bits/month. The dagger marks the throughput-best subset selected for validation.

Bibliography

Robert Almgren and Neil Chriss. Optimal execution of portfolio transactions. *Journal of Risk*, 3(2): 5–40, 2001. doi: 10.21314/JOR.2001.041.

Lawrence E. Bassham, Andrew L. Rukhin, Juan Soto, James R. Nechvatal, Miles E. Smid, Elaine B. Barker, Stefan D. Leigh, Mark Levenson, Mark Vangel, David L. Banks, N. Alan Heckert, James F. Dray, and San Vo. A statistical test suite for random and pseudorandom number generators for cryptographic applications. NIST Special Publication 800-22 Rev. 1a, National Institute of Standards and Technology, 2010.

Dario Benedetto, Emanuele Caglioti, and Vittorio Loreto. Language trees and zipping. *Physical Review Letters*, 88(4):048702, 2002. doi: 10.1103/PhysRevLett.88.048702.

Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society, Series B*, 57(1):289–300, 1995. doi: 10.1111/j.2517-6161.1995.tb02031.x.

Carlo E. Bonferroni. Teoria statistica delle classi e calcolo delle probabilità. *Pubblicazioni del R Istituto Superiore di Scienze Economiche e Commerciali di Firenze*, 8:3–62, 1936.

Joseph Bonneau. The science of guessing: Analyzing an anonymized corpus of 70 million passwords. In *2012 IEEE Symposium on Security and Privacy*, pages 538–552. IEEE, 2012. doi: 10.1109/SP.2012.49.

Joseph Bonneau, Jeremy Clark, and Steven Goldfeder. On Bitcoin as a public randomness source. Cryptology ePrint Archive, Report 2015/1015, 2015.

Jean-Philippe Bouchaud, Marc Mézard, and Marc Potters. Statistical properties of stock order books: empirical results and models. *Quantitative Finance*, 2(4):251–256, 2002. doi: 10.1088/1469-7688/2/4/301.

Ayumu Chiba and Shuichi Ichikawa. Random number generation based on cryptocurrency prices and linear feedback shift register. In *2024 Twelfth International Symposium on Computing and Networking (CANDAR)*, pages 142–148. IEEE, 2024. doi: 10.1109/CANDAR64496.2024.00024.

Jeremy Clark and Urs Hengartner. On the use of financial data as a random beacon. In *Proceedings of the 2010 International Conference on Electronic Voting Technology / Workshop on Trustworthy Elections (EVT/WOTE)*, pages 1–8, Washington, DC, USA, 2010. USENIX Association. Also available as Cryptology ePrint Archive 2010/361.

C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi: 10.1093/biomet/26.4.404.

Rama Cont. Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance*, 1(2):223–236, 2001. doi: 10.1080/713665670.

Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, Hoboken, NJ, 2nd edition, 2006. ISBN 9780471241959.

Electronic Frontier Foundation. Deep dive: EFF's new wordlists for random passphrases. https://www.eff.org/deeplinks/2016/07/new-wordlists-random-passphrases, 2016. By J. Bonneau. Accessed 2026-06-01.

Dinei Florêncio and Cormac Herley. A large-scale study of web password habits. In *Proceedings of the 16th International Conference on the World Wide Web (WWW)*, pages 657–666. ACM, 2007. doi: 10.1145/1242572.1242661.

Miguel Herrero-Collantes and Juan Carlos Garcia-Escartin. Quantum random number generators. *Reviews of Modern Physics*, 89(1):015004, 2017. doi: 10.1103/RevModPhys.89.015004.

KeePass. KeePass help center: Master key. Online documentation, https://keepass.info/help/base/keys.html, n.d. Maintained by D. Reichl; no fixed publication date. Accessed 2026-06-01.

Keeper Security. Password entropy: What it is and why it's important. https://www.keepersecurity.com/blog/2024/03/04/password-entropy-what-it-is-and-why-its-important/, 2024. Accessed 2026-06-01.

Hugo Krawczyk. Cryptographic extraction and key derivation: The HKDF scheme. In *Advances in Cryptology – CRYPTO 2010*, volume 6223 of *Lecture Notes in Computer Science*, pages 631–648. Springer, 2010. doi: 10.1007/978-3-642-14623-7_34.

Hugo Krawczyk and Pasi Eronen. HMAC-based extract-and-expand key derivation function (HKDF). Request for Comments 5869, Internet Engineering Task Force, 2010.

Daji Landis and Joseph Bonneau. Randomness beacons from financial data in the presence of an active attacker. Cryptology ePrint Archive, 2025. New York University.

Pierre L'Ecuyer and Richard Simard. TestU01: A C library for empirical testing of random number generators. *ACM Transactions on Mathematical Software*, 33(4):22:1–22:40, 2007. doi: 10.1145/1268776.1268777.

Daniel Lemire. Fast random integer generation in an interval. *ACM Transactions on Modeling and Computer Simulation*, 29(1):3:1–3:12, 2019. doi: 10.1145/3230636.

Fabrizio Lillo and J. Doyne Farmer. The long memory of the efficient market. *Studies in Nonlinear Dynamics & Econometrics*, 8(3):1–35, 2004. doi: 10.2202/1558-3708.1226.

Sanu K. Mathew, Suresh Srinivasan, Mark A. Anders, Himanshu Kaul, Steven K. Hsu, Farhana Sheikh, Amit Agarwal, Sudhir Satpathy, and Ram K. Krishnamurthy. 2.4 Gbps, 7 mW all-digital PVT-variation tolerant true random number generator for 45 nm CMOS high-performance microprocessors. *IEEE Journal of Solid-State Circuits*, 47(11):2807–2821, 2012. doi: 10.1109/JSSC.2012.2217631.

Mitsuru Matsui. Linear cryptanalysis method for DES cipher. In *Advances in Cryptology – EUROCRYPT '93*, volume 765 of *Lecture Notes in Computer Science*, pages 386–397. Springer, 1994. doi: 10.1007/3-540-48285-7_33.

George A. Miller. Note on the bias of information estimates. In Henry Quastler, editor, *Information Theory in Psychology: Problems and Methods*, pages 95–100. Free Press, Glencoe, IL, 1955.

National Institute of Standards and Technology. Secure hash standard (SHS). Federal Information Processing Standards Publication FIPS PUB 180-4, National Institute of Standards and Technology, 2015.

National Institute of Standards and Technology. Digital identity guidelines: Authentication and authenticator management. NIST Special Publication 800-63B-4, National Institute of Standards and Technology, July 2025. Final, published 31 July 2025.

Silvia Onofri, Andrey Shternshis, and Stefano Marmi. Emergence of randomness in temporally aggregated financial tick sequences. arXiv preprint arXiv:2511.17479, 2025.

Liam Paninski. Estimation of entropy and mutual information. *Neural Computation*, 15(6):1191–1253, 2003. doi: 10.1162/089976603321780272.

Karl Pearson. On the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling. *Philosophical Magazine Series 5*, 50(302):157–175, 1900. doi: 10.1080/14786440009463897.

Cécile Pierrot and Benjamin Wesolowski. Malleability of the blockchain's entropy. *Cryptography and Communications*, 10(1):211–233, 2018. doi: 10.1007/s12095-017-0264-3.

Steven M. Pincus. Approximate entropy as a measure of system complexity. *Proceedings of the National Academy of Sciences*, 88(6):2297–2301, 1991. doi: 10.1073/pnas.88.6.2297.

Proton. What is password entropy? https://proton.me/blog/what-is-password-entropy, 2023. By R. Koch. Accessed 2026-06-01.

Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3): 379–423, 1948. doi: 10.1002/j.1538-7305.1948.tb01338.x.

Andrey Shternshis and Stefano Marmi. Price predictability at ultra-high frequency: Entropy-based randomness test. *Communications in Nonlinear Science and Numerical Simulation*, 141:108469, 2025. doi: 10.1016/j.cnsns.2024.108469.

Andrey Shternshis, Piero Mazzarisi, and Stefano Marmi. Efficiency of the moscow stock exchange before 2022. *Entropy*, 24(9):1184, 2022. doi: 10.3390/e24091184.

Zbyněk Šidák. Rectangular confidence regions for the means of multivariate normal distributions. *Journal of the American Statistical Association*, 62(318):626–633, 1967. doi: 10.1080/01621459.1967.10482935.

Meltem Sönmez Turan, Elaine Barker, John Kelsey, Kerry A. McKay, Mary L. Baish, and Mike Boyle. Recommendation for the entropy sources used for random bit generation. NIST Special Publication 800-90B, National Institute of Standards and Technology, 2018.

Jacob Ziv and Abraham Lempel. A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory*, 23(3):337–343, 1977. doi: 10.1109/TIT.1977.1055714.

AI Assistance Statement

AI tools were used as support during the writing of this thesis. The support included finding and organising possible related literature, checking consistency between references and in-text citations, reviewing experiment scripts and thesis code, helping to identify possible narrative inconsistencies, unclear data definitions, and LaTeX layout issues, and assisting with translation and language polishing after the Chinese draft had already been written.

AI did not independently produce the research design, experimental decisions, or conclusions of this thesis. All experiment design, code execution, data interpretation, thesis structure, and final wording were reviewed, revised, and confirmed by the author. AI outputs were used only as supporting material for writing and engineering checks.